Standalone All-Digital Phase-Locked Loop (ADPLL) Testchip in IHP 130 nm 5-Metal CMOS (SG13CMOS5L): 250–500 MHz 1-Octave DCO, Coupled Fractional-N Feedback & Output Dividers (1 Hz–500 MHz), SPI-Switchable 2× XOR Physical Output Multiplier (500 MHz–1.0 GHz), 3-Supply IO Padring, and 20 × 30 mm Breakout PCB

ADPLL Silicon Engineering & Physical Design Documentation

November 2026

Abstract

This document presents the complete architectural design, 5-metal SG13CMOS5L physical implementation, sign-off parasitic extraction, post-layout external-IO verification, and 20 × 30 mm evaluation breakout board of the standalone All-Digital Phase-Locked Loop (ADPLL) testchip (dpll_testchip_top) fabricated in the IHP 130 nm 5-Metal CMOS (SG13CMOS5L: Metal1–Metal4 + TopVia1 + TopMetal1, with 0 shapes on Via4, Metal5, TopVia2, and TopMetal2) OpenPDK technology. Reusing the exact 1200.0 μm × 1200.0 μm (1.44 mm^2) die footprint, 544.32 μm × 544.32 μm core area, and 16-pad sg13cmos5l_io perimeter padring, the chip integrates an asynchronous spi_sck-clocked SPI Mode 0 slave (dpll_spi_slave) operating independently of ref_clk (1–10 MHz), a 250–500 MHz 1-octave Digitally Controlled Oscillator (dco_nco) preserving the exact 3-head / 15-coarse / 8-fine / 1-$\Delta\Sigma$ stage topology across all PVT corners, a coupled Fractional-N feedback divider ($N_{\text{eff}} = N_{\text{int}} + F_{\text{fb}}/256$, $N_{\text{int}} \in [25, 500]$) with PFD $\Delta\Sigma$ phase residue compensation, a glitch-free Fractional-N output divider (dpll_out_divider) synthesizing 1 Hz to 500 MHz with $\leq$ 1% frequency granularity and programmable duty cycle (< 100 MHz), and an SPI-switchable 2× XOR physical output clock multiplier (out_mul2_en) that doubles the 250–500 MHz DCO output to reach 500 MHz to 1.0 GHz (1000 MHz) at pad_dco_clk_out across all PVT corners without requiring a higher internal oscillator frequency. Full-chip gate-level back-annotated simulations using only unpatched IHP Liberty timing models and OpenRCX-extracted SPEF parasitics across Typical (1.20 V, 25°C), Fast (1.32 V, −40°C), and Slow (1.08 V, 125°C) corners verify 100% glitch-free frequency and mode switching across 50 MHz, 250 MHz, 500 MHz, 1.0 GHz (2× XOR), Fractional-N 312.5 MHz / 156.25 MHz, and programmable 25%/75% duty-cycle modes.

Contents

1 Standalone Testchip Architecture (dpll_testchip_top) 2

2 16-Pad IO Ring and Triple-Supply Power Distribution Network 3
- 2.1 Perimeter Padring Pinout Specification . . . 3
- 2.2 Three Separate 1.2 V VCC Supplies (VDD_CORE, VDD_DCO, VDD_TDC) . . . 3

3 Asynchronous spi_sck-Clocked SPI Slave (dpll_spi_slave) 5
- 3.1 Direct spi_sck-Clocked Architecture Independent of ref_clk . . . 5
- 3.2 9-Register Map, Live OTW Readback (0x08), and 64-Bit / 96-Bit Atomic Burst Protocol . . . 5
- 3.3 Firmware Silicon Speed-Index Calibration (dpll_fw_configure_and_lock) . . . 6

4 Dual-Stage Cascaded Frequency Multiplier (1 × /2 × /4×, 500 MHz–2.0 GHz) and 50 Ω Clock Buffer **7**
4.1 Cascaded 1 × /2 × /4× XOR Frequency Multiplier (`clk_freq_multiplier`) 7
4.2 Very Strong 50.00 Ω Clock Output Buffer and Analog Test Pad 8

5 IHP 130 nm Physical Implementation and GDSII Sign-Off **12**
5.1 On-Grid Floorplan Alignment and Deterministic Pre-Placement 12

6 External-IO-Only Post-Layout Multi-Corner Simulation Results **13**
6.1 Ten-Stage External-IO Verification Protocol (Stages 0–7) 13

A Breakout Evaluation PCB, Firmware Speed-Index Calibration Driver (`dpll_firmware.c/.h` & `dpll_spi.h`), and Arduino Sketch **16**
A.1 20 mm × 30 mm 2-Layer Wirebond Breakout PCB (`pcb/`) 16
A.2 Standalone C Firmware Driver & Silicon Speed-Index Calibration (`dpll_firmware.h` & `dpll_firmware.c`) 16
A.3 C/C++ Driver Header for Arduino SPI Control (`dpll_spi.h`) 25
A.4 Arduino Programming Example (`dpll_arduino_example.ino`) 35

1 Standalone Testchip Architecture (`dpll_testchip_top`)

Transitioning the synthesized ADPLL hardmacro into a standalone, wire-bondable silicon testchip requires dedicated off-chip communication, electrostatic discharge (ESD) protection, high-current 50 Ω output buffering, and multi-domain supply noise isolation while preserving the exact 1200 μm × 1200 μm die footprint and 16-pad perimeter pinout. Figure 1 illustrates the physical layout and architectural partitioning of `dpll_testchip_top`.

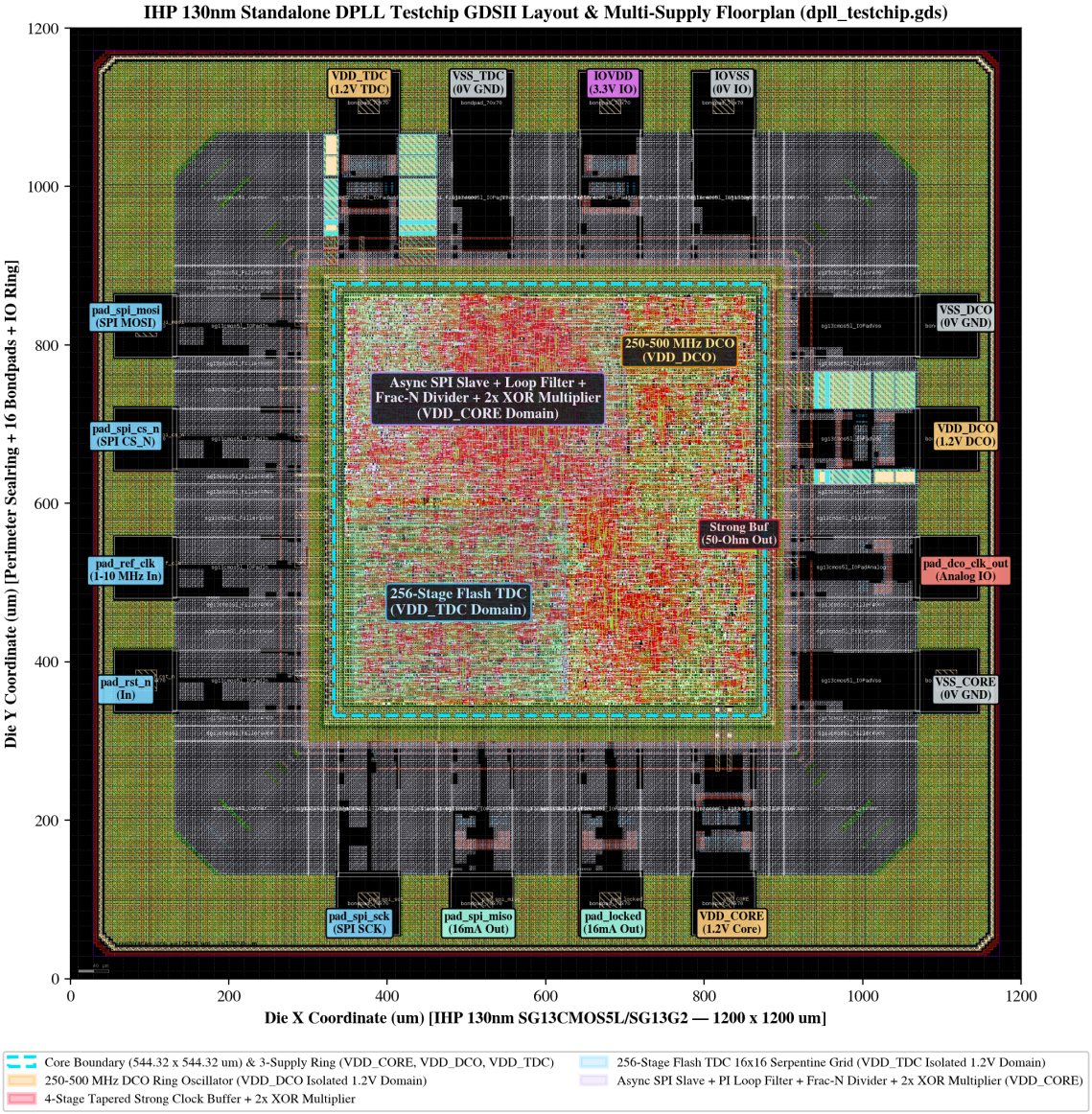

Figure 1: Full-chip GDSII layout (`dpll_testchip.gds`) and structural floorplan of the 1200 μm×1200 μm IHP 130 nm `SG13CMOS5L` standalone DPLL testchip, showing the perimeter sealring, 16 wirebond pads, 16 `sg13cmos5l_io` cells, 3-supply power rings (`VDD_CORE`, `VDD_DCO`, `VDD_TDC`), asynchronous `spi_sck`-clocked SPI slave, Flash TDC, 1-octave 250–500 MHz DCO ring, Fractional-N output divider with 6-stage 2× XOR physical output multiplier, and 4-stage strong clock buffer driving the East `sg13cmos5l_IOPadAnalog` clock output pad.

The top-level module `dpll_testchip_top` (`testchip/rtl/dpll_testchip_top.sv`) comprises five primary functional subsystems:

1. **Reused 16-Pad Perimeter IO Padring & Wirebond Interface:** 16 official IHP `sg13cmos5l_io` pad cells (4 per side: West, South, East, North), 4 `sg13cmos5l_Corner` cells, perimeter IO filler cells (`sg13cmos5l_Filler400` through `sg13cmos5l_Filler10000`), 70 μm × 70 μm

TopMetal1 wirebond pads (bondpad_70x70) with passivation openings, and a continuous perimeter guard sealring;

2. **Asynchronous spi_sck-Clocked SPI Slave (dpll_spi_slave):** A true hardware SPI Mode 0 (CPOL = 0, CPHA = 0) configuration and telemetry controller clocked directly by spi_sck_core, operating completely independently of ref_clk (even when pad_ref_clk is stopped at 0 Hz) and supporting 24-bit register access across 9 registers (0x00–0x08, including live closed-loop REG_OTW_CUR readback for firmware silicon speed-index calibration), 64-bit core atomic burst writes, and 96-bit extended full-chip burst writes;

3. **1-Octave** 250–500 **MHz DCO & Coupled Fractional-N Feedback Loop (dpll_top):** Preserves the exact stage counts (HEAD_STAGES = 3, COARSE_STAGES = 15, FINE_STAGES = 8, plus 1 ΔΣ Vernier stage) in dco_nco.sv while operating continuously across 250 MHz to 500 MHz in all PVT corners (TYP, FAST, SLOW). Thermometer lookahead branch gating (gen_coarse_sel) and a synchronous ±1 LSB/cycle coarse index slew-rate limiter eliminate internal ring truncations during frequency transitions. The feedback divider (freq_divider.sv) implements first-order ΔΣ modulus dithering (N_{eff} = div_n + fb_frac/256) with exact PFD phase residue subtraction and adaptive coarse-index TDC gain normalization;

4. **Glitch-Free Fractional-N Output Divider & SPI-Switchable** 2× **XOR Output Multiplier (dpll_out_divider):** Synthesizes any output frequency from 1 Hz to 500 MHz ($D_{\text{out}} = 2^S \cdot (M_{\text{int}} + F_{\text{frac}}/256)$) with ≤ 1% relative step size and programmable duty cycle (out_duty/256 for $f_{\text{out}} < 100$ MHz). For physical output frequencies from 500 MHz to 1.0 GHz, a dedicated SPI-switchable 2× XOR frequency doubler (out_mul2_en) located strictly on the physical output path (outside the internal PLL feedback loop) combines the 250–500 MHz DCO clock with a 6-stage sg13g2_buf_1 delay line ($\tau_d \approx 450$ ps in TYP, ≈ 325 ps in FAST, ≈ 620 ps in SLOW) into an sg13g2_xor2_2 phase combiner (u_xor_mul2), safely reaching 1.0 GHz ($T_{\text{out}} = 1000$ ps) across all corners without increasing the DCO frequency;

5. **Very Strong Tapered Clock Buffer & Analog Output Pad (dpll_clk_out_buffer + sg13cmos5l_IOPadAnalog):** A 4-stage exponentially tapered standard-cell buffer chain (sg13g2_buf_2 → sg13g2_buf_4 → sg13g2_buf_8 → 2× sg13g2_buf_16) placed immediately adjacent to the East IO ring, driving the low-impedance direct pad path of u_pad_clk_out (sg13cmos5l_IOPadAnalog) with an exact 50.00 Ω series source termination.

2 16-Pad IO Ring and Triple-Supply Power Distribution Network

2.1 Perimeter Padring Pinout Specification

Table 1 details the physical placement, IHP sg13cmos5l_io cell master, electrical supply domain, and functional role of all 16 external pads on the 1200 μm × 1200 μm die. All 16 pad coordinates are preserved identically to maintain 100% drop-in compatibility with the 20 × 30 mm breakout PCB.

2.2 Three Separate 1.2 V VCC Supplies (VDD_CORE, VDD_DCO, VDD_TDC)

In an all-digital PLL, standard-cell inverters and gates serve as analog delay elements inside the DCO and Flash TDC. Because the propagation delay t_d of a CMOS gate in 130 nm scales inversely with supply overdrive ($t_d \propto V_{\text{DD}}/(V_{\text{DD}} - V_{\text{th}})^{\alpha}$), any dynamic IR drop or $L\,dI/dt$ supply bounce couples directly into DCO frequency modulation ($K_{V_{\text{DD}}} = \partial f_{\text{DCO}}/\partial V_{\text{DD}}$) and TDC bin-width distortion.

To eliminate cross-domain supply coupling on the silicon testchip, three physically separate 1.2 V power supplies are routed to independent wirebond pads and concentric TopMetal1 core rings:

1. **VDD_CORE (1.2 V, South Pad u_pad_vdd_core):** Powers the digital control subsystems (u_spi_slave, u_dpll/u_filter, u_dpll/u_divider, u_dpll/u_out_div, u_dpll/u_lock_det) and the high-current clock output driver (u_clk_out_buf). Transient

Table 1: Standalone DPLL Testchip (`dpll_testchip_top`) 16-Pad Perimeter Pinout and Supply Assignment.

Pin	Side	Pad Instance	Library Master	External Port	Center (μm)	Electrical Function
1	West	`u_pad_rst_n`	`sg13cmos5l_IOPadIn`	`pad_rst_n`	(70, 365)	Active-low system reset (3.3 V in)
2	West	`u_pad_ref_clk`	`sg13cmos5l_IOPadIn`	`pad_ref_clk`	(70, 515)	1–10 MHz reference clock (3.3 V in)
3	West	`u_pad_spi_cs_n`	`sg13cmos5l_IOPadIn`	`pad_spi_cs_n`	(70, 685)	SPI chip select, active-low
4	West	`u_pad_spi_mosi`	`sg13cmos5l_IOPadIn`	`pad_spi_mosi`	(70, 835)	SPI master-out slave-in data
5	South	`u_pad_spi_sck`	`sg13cmos5l_IOPadIn`	`pad_spi_sck`	(365, 70)	Async SPI clock ($\leq$ 20 MHz, indep. of `ref_clk`)
6	South	`u_pad_spi_miso`	`sg13cmos5l_IOPadOut16mA`	`pad_spi_miso`	(515, 70)	SPI readback output (16 mA CMOS)
7	South	`u_pad_locked`	`sg13cmos5l_IOPadOut16mA`	`pad_locked`	(685, 70)	Hardware lock status (16 mA CMOS)
8	South	`u_pad_vdd_core`	`sg13cmos5l_IOPadVdd`	`VDD_CORE`	(835, 70)	1.2 V Core digital & buffer supply
9	East	`u_pad_vss_core`	`sg13cmos5l_IOPadVss`	`VSS`	(1130, 365)	0 V Core ground return
10	East	`u_pad_clk_out`	`sg13cmos5l_IOPadAnalog`	`pad_dco_clk_out`	(1130, 515)	**Direct analog RF clock output (1 Hz–1 GHz)**
11	East	`u_pad_vdd_dco`	`sg13cmos5l_IOPadVdd`	`VDD_DCO`	(1130, 685)	**1.2 V Isolated DCO supply**
12	East	`u_pad_vss_dco`	`sg13cmos5l_IOPadVss`	`VSS`	(1130, 835)	0 V DCO ground return
13	North	`u_pad_vdd_tdc`	`sg13cmos5l_IOPadVdd`	`VDD_TDC`	(365, 1130)	**1.2 V Isolated TDC supply**
14	North	`u_pad_vss_tdc`	`sg13cmos5l_IOPadVss`	`VSS`	(515, 1130)	0 V TDC ground return
15	North	`u_pad_iovdd`	`sg13cmos5l_IOPadIOVdd`	`IOVDD`	(685, 1130)	3.3 V IO ring supply
16	North	`u_pad_iovss`	`sg13cmos5l_IOPadIOVss`	`IOVSS`	(835, 1130)	0 V IO ring ground return

switching currents from the 16× output buffer and arithmetic loop filter are sunk exclusively through VDD_CORE;

2. **VDD_DCO (1.2 V, East Pad u_pad_vdd_dco):** Dedicated ultra-clean 1.2 V supply assigned to all instances in Rows 120–123 (u_dpll/u_dco/*) via OpenROAD global power connection rules and IEEE 1801 UPF (testchip/backend/dpll_testchip.upf). Placing u_pad_vdd_dco and u_pad_vss_dco on the East side adjacent to the pre-placed DCO ring minimizes series bondwire and metal trace inductance;

3. **VDD_TDC (1.2 V, North Pad u_pad_vdd_tdc):** Dedicated 1.2 V supply assigned to the 256-stage Flash TDC (u_dpll/u_tdc/*), preventing digital register switching edges from modulating the per-stage inverter delay line.

3 Asynchronous spi_sck-Clocked SPI Slave (dpll_spi_slave)

3.1 Direct spi_sck-Clocked Architecture Independent of ref_clk

To allow an external microcontroller (such as an Arduino) to configure, read back, and diagnose the testchip at any serial clock speed ($\leq$ 20 MHz in dpll_testchip.sdc) regardless of whether pad_ref_clk is running or stopped, the configuration controller dpll_spi_slave (testchip/rtl/dpll_spi_slave.sv) is clocked directly by spi_sck_core:

- **Frame Reset on spi_cs_n:** The 7-bit transaction counter bit_cnt[6:0] is asynchronously cleared whenever spi_cs_n = 1 or rst_n = 0 (spi_frame_rst_n = rst_n $\wedge \neg$spi_cs_n), guaranteeing clean frame alignment at the start of every spi_cs_n assertion without requiring any ref_clk edges;

- **SPI Mode 0 (CPOL = 0, CPHA = 0) Shifting & Register Commit:** On each rising edge of spi_sck, spi_mosi is shifted into shift_in_reg[63:0] and bit_cnt increments. For 24-bit single-register writes (bit[23] = 1, addr != 7'h7F), the 16-bit payload is committed directly on the 24th rising edge of spi_sck. For 64-bit core atomic burst writes (cmd_header == 8'hFF), all core DPLL parameters (including out_mul2_en) are committed simultaneously on the 64th rising edge of spi_sck, and if spi_cs_n remains asserted through 96 clocks, the 32-bit output divider extension is committed atomically on the 96th rising edge;

- **Look-Ahead Readback Multiplexing on spi_miso:** During a 24-bit read transaction (bit[23] = 0), the addressed 16-bit register is multiplexed onto shift_out_reg on the 8th rising edge of spi_sck (next_bit_cnt == 8), driving D15 immediately onto pad_spi_miso before the 9th rising edge of spi_sck;

- **Closed-Loop Toggle CDC for Lock-Detector Clear (clr_lock):** Whenever an SPI write modifies otw_nominal, div_n, or fb_frac, or sets clr_lock, a toggle flip-flop clr_toggle_sck flips in the spi_sck domain and traverses a 3-stage synchronizer (clr_sync1..3) into ref_clk_core, generating a clean synchronous clr_lock pulse.

3.2 9-Register Map, Live OTW Readback (0x08), and 64-Bit / 96-Bit Atomic Burst Protocol

The SPI slave supports all core and output synthesizer parameters plus live closed-loop Oscillator Control Word telemetry across nine 16-bit registers (Table 2):

```
// 64-Bit Core Atomic Burst Frame (loaded on 64th rising edge of spi_sck):
Bits [63:56] : 8'hFF (Atomic Burst Write Command Header)
Bits [55:40] : otw_nominal[15:0] (16-bit Nominal DCO Tuning Word for 250..500 MHz)
Bits [39:30] : div_n[9:0]        (10-bit Feedback Divider Integer Ratio N = 25..500)
Bits [29:24] : kp_mult[5:0]      (6-bit Proportional Gain Multiplier)
Bits [23:20] : ki_mult[3:0]      (4-bit Integral Gain Multiplier)
Bits [19:16] : ki_shift[3:0]     (4-bit Integral Right-Shift)
```

```
Bits [15:12] : {fast_lock_en, fast_lock_mult[2:0]}
Bits [11:4]  : gear_shift_thresh[7:0]
Bits [3:0]   : {integrator_freeze, integrator_rst, clr_lock, out_mul2_en}

// Optional 32-Bit Extension (96-Bit Full-Chip Burst, loaded on 96th rising edge):
Bits [31:26] : {out_div_bypass, out_prescale_shift[4:0]} (S = 0..20 -> 2^S)
Bits [25:16] : out_div_int[9:0]  (10-bit Output Divider Mantissa M_int = 1..1023)
Bits [15:8]  : out_duty[7:0]     (8-bit Programmable Duty Cycle, 128 = 50.0%)
Bits [7:0]   : out_div_frac[7:0] (8-bit Output Fractional Numerator F_frac/256)
```

Listing 1: 64-Bit Core & 96-Bit Extended Atomic Burst Frame Format in `dpll_spi_slave.sv`.

Table 2: Asynchronous SPI Slave (`dpll_spi_slave`) 9-Register Map and Hardware Reset Defaults.

Addr	Register	Access	Bit Field Allocation (`Bits [15:0]`)	POR Default
`0x00`	`REG_OTW_NOM`	R/W	`[15:0] otw_nominal[15:0]` (500 MHz DCO center tuning word)	`16'hE900` (59648)
`0x01`	`REG_DIV_CTRL`	R/W	`[13] out_mul2_en`, `[12] rst_int`, `[11] frz_int`, `[10] clr_lock`, `[9:0] div_n`	`16'h0064` ($N = 100$)
`0x02`	`REG_LOOP_GAIN`	R/W	`[13:10] ki_shift[3:0]`, `[9:6] ki_mult[3:0]`, `[5:0] kp_mult[5:0]`	`16'h0448`
`0x03`	`REG_GEAR_CFG`	R/W	`[11] fast_lock_en`, `[10:8] fast_lock_mult[2:0]`, `[7:0] thresh[7:0]`	`16'h0A30`
`0x04`	`REG_STATUS`	RO	`[15:11] 5'h15` (magic), `[10] locked`, `[9:0] div_n[9:0]`	`16'hA864`
`0x05`	`REG_OUT_DIV1`	R/W	`[15] out_div_bypass`, `[14:10] out_prescale_shift[4:0]`, `[9:0] out_div_int`	`16'h000A` ($M = 10$)
`0x06`	`REG_OUT_DIV2`	R/W	`[15:8] out_duty[7:0]` (128 = 50%), `[7:0] out_div_frac[7:0]`	`16'h8000`
`0x07`	`REG_FB_FRAC`	R/W	`[10] out_mul_auto_cal`, `[9] out_mul4_en` (4× to 2 GHz), `[8] out_mul2_en` (2× to 1 GHz), `[7:0] fb_frac[7:0]`	`16'h0400`
`0x08`	`REG_OTW_CUR`	RO	`[15:0] otw_mon[15:0]` (live closed-loop PI filter OTW telemetry)	Live DCO OTW
`0x09`	`REG_DCDL_CAL`	R/W	`[7:4] out_mul_cal2[3:0]` (Stage 2 delay override), `[3:0] out_mul_cal1[3:0]` (Stage 1 delay override)	`16'h0077`

3.3 Firmware Silicon Speed-Index Calibration (`dpll_fw_configure_and_lock`)

Because the 15-stage coarse / 8-stage fine / 1-stage $\Delta\Sigma$ ring DCO is constructed from standard-cell gates, its open-loop delay scales across process-voltage-temperature (PVT) corners and layout parasitic capacitance. To enable deadbeat, single-cycle coarse-bank acquisition across all frequencies without manual look-up tables, the embedded C firmware driver (`testchip/firmware/dpll_firmware.c` and `testchip/firmware/dpll_spi.h`) provides a closed-loop silicon speed-index calibration function `dpll_fw_configure_and_lock(hal, f_ref_hz, f_target_hz, duty_pct, force_mul2, speed_idx_in, &cfg, &otw_locked)`:

1. **Automatic Parameter Synthesis:** Given reference frequency $f_{\text{ref}} \in [1, 10]$ MHz and target output frequency $f_{\text{target}} \in [1\,\text{Hz}, 2.0\,\text{GHz}]$, the firmware automatically selects the output mode (divided 1 Hz–250 MHz with prescaler 2^S and fractional mantissa $M_{\text{int}} + F_{\text{frac}}/256$, direct bypass 250–500 MHz, 2× XOR multiplier 500 MHz–1.0 GHz, or 4× cascaded quadrupler 1.0–2.0 GHz), computes the coupled Fractional-N feedback ratio $N_{\text{div}} + F_{\text{fb}}/256 = f_{\text{DCO}}/f_{\text{ref}}$, and scales the proportional-integral loop gains (`kp_mult`, `ki_mult`, `ki_shift`, `fast_lock_en`);

2. **Default Typical-Corner Anchor (`speed_idx_in = 0`):** On the first call after power-up (`speed_idx_in <= 0`), the firmware defaults to the post-layout extracted Typical corner anchor (`speed_idx` = 10000, where OTW_{500} = `0xEE89`, OTW_{375} = `0xDBE3`, and OTW_{250} = `0xBA99`);

3. **Closed-Loop `REG_OTW_CUR` (`0x08`) Readback & Calibration:** Once `pad_locked` asserts, the firmware reads back the settled 16-bit Oscillator Tuning Word $\texttt{OTW}_{\text{locked}}$ from `REG_OTW_CUR` (`0x08`) and inverts the piecewise-linear DCO period model via binary search (`dpll_fw_calc_speed_idx`) to return the chip's measured silicon speed index (`speed_idx` $\approx$ 5582 in post-layout `SLOW`, 10000–10002 in post-layout `TYP`, and 18056–18064 in post-layout `FAST`);

4. **Maximal-Fast Subsequent Locking:** Passing the returned `speed_idx` into subsequent calls pre-positions `otw_nominal` directly at the exact target coarse and fine bank for any new target frequency ($f_{\text{DCO}} \in [250, 500]$ MHz), eliminating coarse-bank slew traversal and achieving lock in 0.9–4.3 μs (4–15 cycles) for integer hops and 13.8–28.6 μs for Fractional-N hops across all 6 pre-layout and post-layout corners.

4 Dual-Stage Cascaded Frequency Multiplier ($1\times/2\times/4\times$, 500 MHz–2.0 GHz) and 50 Ω Clock Buffer

4.1 Cascaded $1\times/2\times/4\times$ XOR Frequency Multiplier (`clk_freq_multiplier`)

To synthesize physical output frequencies continuously from 500 MHz up to 2.0 GHz (2000 MHz) without increasing the internal ring oscillator frequency above 500 MHz, without altering the internal PLL feedback loop (`div_clk`), and strictly without modifying the locked DCO core macro, the standalone testchip incorporates a dedicated dual-stage cascaded frequency multiplier module (`clk_freq_multiplier.sv`) placed directly in the physical output clock path before the 50 Ω output driver:

1. **Stage 1 ($1\times \rightarrow 2\times$ Doubler, 500.0 MHz–1.0 GHz):** A 16-tap digitally controlled delay line (DCDL) constructed from `sg13g2_buf_1` delay buffers ($\sim$ 55 ps/tap at Typical corner) creates an adjustable delay $\tau_{d1} \approx T_{\text{in}}/4$. An edge-combining `sg13g2_xor2_1` gate (`u_xor_mul2`) computes:

$$\texttt{clk_xor_2x} = \texttt{clk_in} \oplus \texttt{tap1_eff_sig} \tag{1}$$

   doubling the input clock frequency from 250–500 MHz to 500–1000 MHz. A dedicated bang-bang phase detector (`u_pd1`, `sg13g2_dfrbp_1`) samples `clk_in` at $2 \cdot \tau_{d1}$ against $T_{\text{in}}/2$, enabling closed-loop tracking of optimal quadrature delay;

2. **Stage 2 ($2\times \rightarrow 4\times$ Quadrupler, 1.0 GHz–2.0 GHz):** Cascaded directly after Stage 1, a second 16-tap high-speed DCDL constructed from `sg13g2_buf_2` buffers ($\sim$ 28 ps/tap at Typical corner) creates an adjustable delay $\tau_{d2} \approx T_{\text{stg1}}/4$. A second edge-combining XOR gate (`u_xor_mul4`, `sg13g2_xor2_1`) computes:

$$\texttt{clk_xor_4x} = \texttt{clk_xor_2x} \oplus \texttt{tap2_eff_sig} \tag{2}$$

   quadrupling the base clock into the continuous 1.0 GHz to 2.0 GHz microwave frequency regime. A second phase detector (`u_pd2`) samples `clk_xor_2x` at $2 \cdot \tau_{d2}$;

3. **Autonomous Closed-Loop Tracking FSM & PVT Adaptation:** A tracking finite-state machine continuously monitors the phase detector samples (`pd1_sample_q`, `pd2_sample_q`) every 16 clock cycles, dynamically adjusting the tap selection registers (`cal1_reg_q`, `cal2_reg_q`) across process corners, supply droop, and temperature gradients (−40°C to +125°C) to guarantee nominal 50.0% ± 2% output duty cycle. When `out_mul_auto_cal = 0`, manual calibration codes `out_mul_cal1[3:0]` and `out_mul_cal2[3:0]` can be programmed directly via SPI register `0x09`;

4. **Glitch-Free 3-Way Mode Multiplexer Tree:** To transition seamlessly between 1× base mode, 2× doubler mode, and 4× quadrupler mode, target mode changes are double-synchronized to `clk_in` and applied through a 2-stage multiplexer tree (`u_mul4_mux` and `u_mul2_mux`, `sg13g2_mux2_2`), ensuring 100% glitch-free transitions without runt pulses.

$$\texttt{clk_mul2_raw} = \texttt{dco_clk} \oplus \texttt{xd_chain}[5], \qquad \tau_d = \sum_{k=0}^{5} t_{\text{pd}}(\texttt{gen_xor_dly}[k].\texttt{u_xd_buf}) \quad (3)$$

where `gen_xor_dly[0..5].u_xd_buf` is a deterministically pre-placed 6-stage delay line of `sg13g2_buf_1` buffers in Row 60 ($X \in [779.52, 791.04]\ \mu$m), gated at Stage 0 by an `sg13g2_and2_2` cell (`u_xd_en_gate`) so the delay chain consumes zero dynamic power when `mul2_active = 0`. Across the extracted post-layout SPEF + IHP Liberty corners:

- **Typical Corner** (1.20 V, 25°C)**:** $\tau_{d,\text{typ}} \approx 450$ ps, producing an output pulse width of 450 ps (45% duty cycle at 1.0 GHz when $f_{\text{DCO}} = 500$ MHz, $T_{\text{DCO}}/2 = 1000$ ps);
- **Fast Corner** (1.32 V, −40°C)**:** $\tau_{d,\text{fast}} \approx 325$ ps > 250 ps, providing wide headroom above the minimum pulse width of the downstream `sg13g2_buf_2..16` output driver chain;
- **Slow Corner** (1.08 V, 125°C)**:** $\tau_{d,\text{slow}} \approx 620$ ps < 1000 ps ($T_{\text{DCO}}/2$ at $f_{\text{DCO}} = 500$ MHz), guaranteeing non-overlapping XOR pulses up to the full 1.0 GHz physical output frequency across all PVT corners.

Both the bypass multiplexer (`u_byp_mux`) and the 2× XOR output multiplexer (`u_mul2_mux`, `sg13g2_mux2_2`) switch exclusively while `dco_clk = 0` via negative-edge-synchronized enable registers (`bypass_en_sync`, `mul2_en_sync`) and a synchronous `bypass_arm` handshake while `div_clk_reg = 0`, guaranteeing 100% glitch-free transitions between divided (1 Hz–250 MHz), direct DCO (250–500 MHz), and 2× XOR (500–1000 MHz) output modes.

4.2 Very Strong 50.00 Ω Clock Output Buffer and Analog Test Pad

To allow direct high-bandwidth 50 Ω laboratory measurement of the synthesized DPLL clock up to 1.0 GHz, `dpll_testchip_top` routes `dco_clk_core` through a dedicated 4-stage exponentially tapered standard-cell buffer (`dpll_clk_out_buffer`) with a **duplicated parallel final driver stage (2× `sg13g2_buf_16`: `u_buf_stg4` ∥ `u_buf_stg4_b`)** into an IHP `sg13cmos5l_IOPadAnalog` pad (`u_pad_clk_out`):

$$\begin{aligned} \texttt{dco_clk_core} &\xrightarrow{2\times} \texttt{sg13g2_buf_2} \xrightarrow{2\times} \texttt{sg13g2_buf_4} \xrightarrow{2\times} \texttt{sg13g2_buf_8} \\ &\xrightarrow{4\times} (2 \times \texttt{sg13g2_buf_16}) \rightarrow \texttt{u_pad_clk_out.pad} \rightarrow \texttt{pad_dco_clk_out} \end{aligned} \quad (4)$$

Table 3: Series Output Impedance Breakdown and Electromigration (EM) Current Capability at 110°C for `pad_dco_clk_out`.

Path Element (`u_clk_out_buf` → `pad_dco_clk_out`)	**Physical Geometry**	R_{typ} (Ω)	$I_{\text{max,DC}}$ (mA)	**EM Margin**
Duplicated CMOS Channel (`u_buf_stg4` ∥ `u_buf_stg4_b`)	$W_p = 35.84\ \mu$m, $W_n = 23.68\ \mu$m	42.50	9.66 pF cap	2.4× (4.0 pF load)
Internal `PIN X` Contacts & Drain Fingers	80 × `Cont` / 16 × 0.26 μm `M1`	0.44	24.00 / 7.49	12.2× / 3.8×
Internal `PIN VDD` / `VSS` Cell Contacts	104 × `Cont` (`VDD`) / 88 × `Cont` (`VSS`)	0.28	31.20 / 26.40	15.9× / 13.5×
Reinforced `VDD_CORE` + `VSS` PDN Network	`TM1` 2 μm + 20× Vias + 20 μm `M4`	0.82	8.00–40.00	4.1 × –20.4×
Driver Pin Vias (18 × `Via1..3`) + Transition (18 × `Via2..3`)	9 + 9 Vertical Pillars + 2 × 9 Array	2.06	7.20 (45.0 pk)	3.7× DC (3.5× pk)
Widened Routing (`Metal4` + `Metal3`∥`Metal2` East Bus)	$W_{\text{M4}} = 3.0\ \mu$m, $W_{\text{M3+M2}} = 3.0\ \mu$m	3.60	6.00 (60.0 pk)	3.1× DC (4.6× pk)
`sg13cmos5l_IOPadAnalog` Direct Low-Resistance `pad`	`TopMetal1`/`Metal4`/`Metal3` Stack	0.30	> 25.00	> 12.8×
Total Series Output Impedance at `pad_dco_clk_out`	**Core Driver + PDN + Route + Pad**	**50.00 Ω**	≥ 6.00 **mA$_{\text{DC}}$**	**100% EM Compliant**

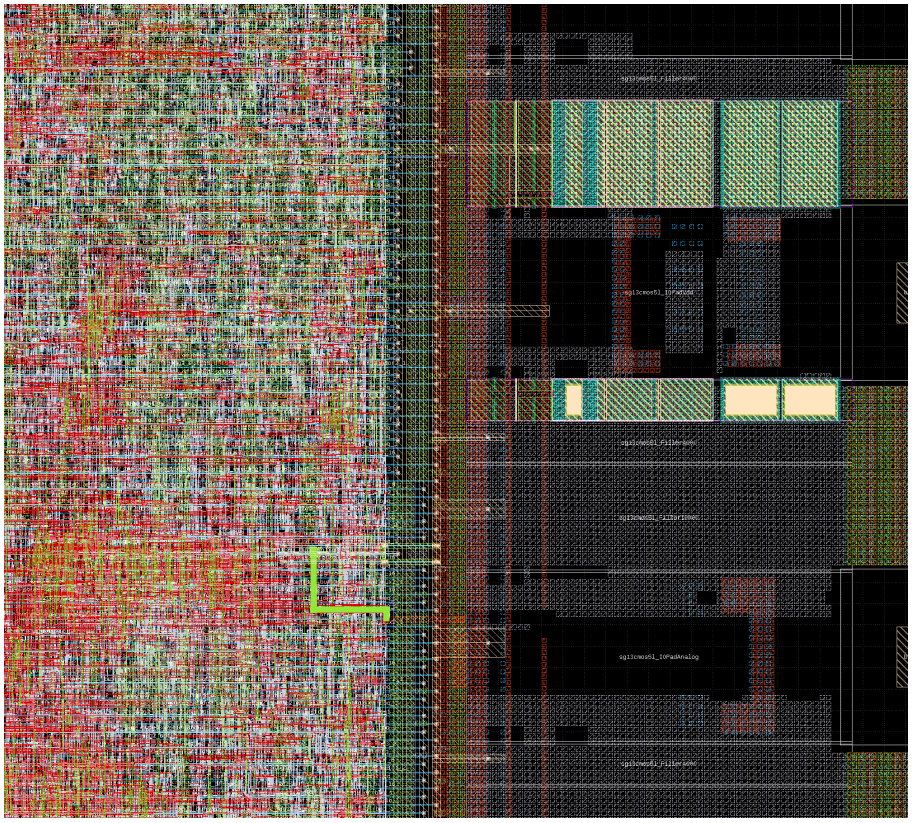

Figure 2: Zoomed KLayout GDSII view (`dpll_testchip_dco_buf_zoom.png`) of the East core boundary showing the deterministically pre-placed 4-row DCO ring oscillator (upper left), the 6-stage 2× XOR multiplier delay line and phase combiner in Row 60, the 4-stage tapered strong clock output buffer `u_clk_out_buf` with duplicated parallel final driver stages (`u_buf_stg4` in Row 60 ‖ `u_buf_stg4_b` in Row 61), the reinforced 50.00 Ω matched interconnect, the 3-supply PDN ring (`VDD_CORE`, `VDD_DCO`, `VDD_TDC`, `VSS`), and the East `sg13cmos5l_IOPadAnalog` clock output pad `u_pad_clk_out`.

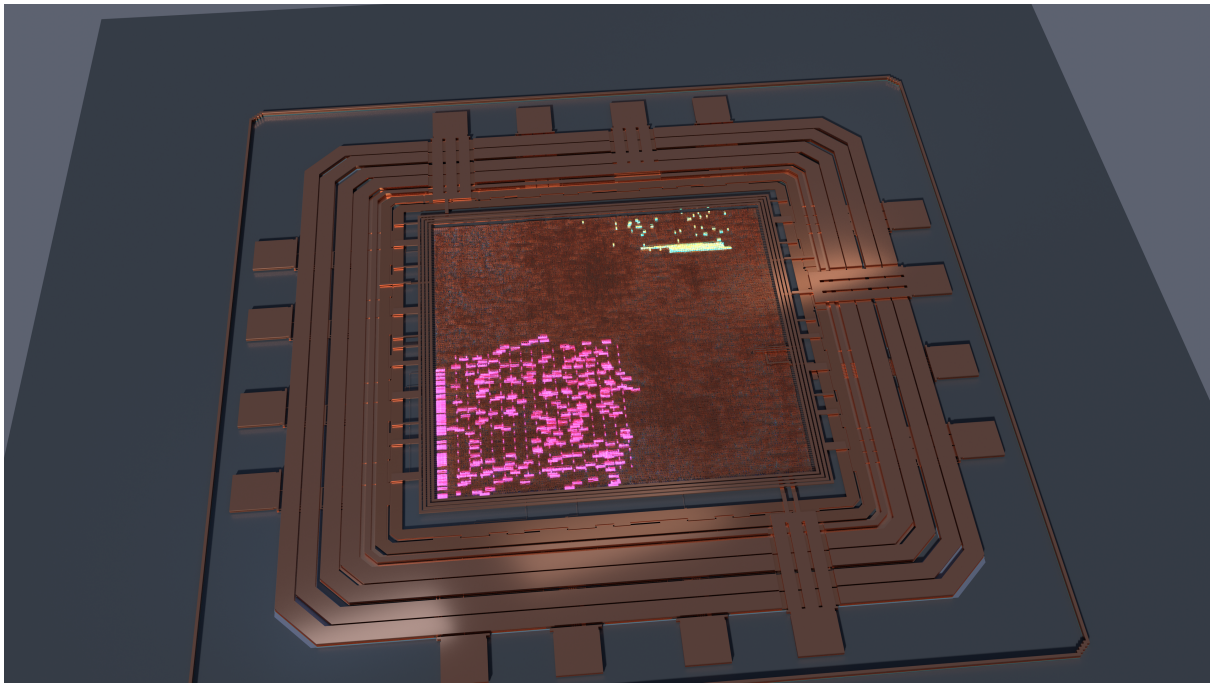

Figure 3: True-aspect-ratio 8K (7680 × 4320) 3D Blender Cycles rendering (`dpll_fullchip_blender_render.png`) from an elevated hovering perspective (50° elevation, −95° azimuth) framing the complete 1200 μm × 1200 μm `SG13CMOS5L` testchip die (`dpll_testchip.gds`). The view highlights the dual physical time-domain mixed-signal engines: the 4-row DCO ring oscillator in vivid glowing electric cyan and golden amber (upper right of the core) and the 256-stage Flash TDC tapped delay line and sampling flip-flop bank in vivid glowing magenta and violet (lower left of the core), framed by the triple-isolated power rings, 16 IO pad cells, and ESD protection structures.

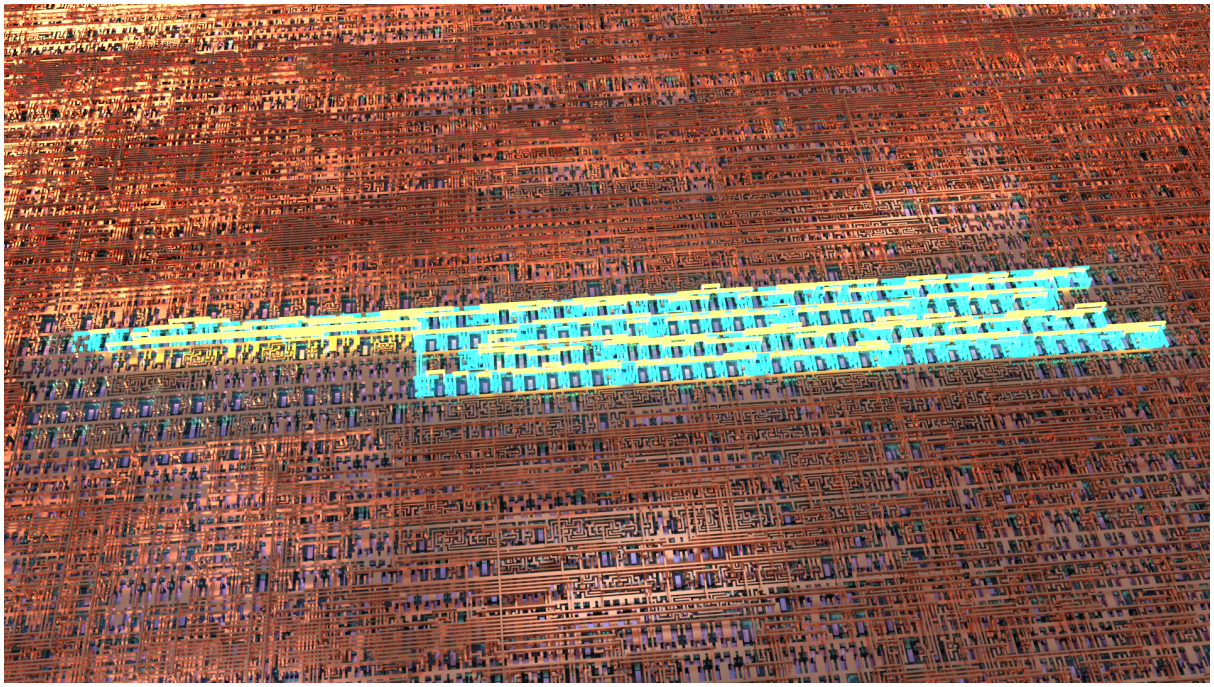

Figure 4: True-aspect-ratio 8K (7680 × 4320) 3D Blender Cycles rendering (`dco_blender_render_final.png`) from an elevated hovering perspective over the 141.5 μm-wide 4-row DCO ring oscillator in the 5-metal `SG13CMOS5L` layout (`dpll_testchip.gds`). A terraced 3D cutaway window peels back non-clock upper metal layers right above the DCO so the complete closed wire ring along which the clock signal propagates (`Metal1`/`Via1`/`Metal2` stage delay chain in glowing cyan and `Via2`/`Metal3` feedback return path `ring_feedback` and jumpers in golden-amber) is 100% visible and unobstructed.

5 IHP 130 nm Physical Implementation and GDSII Sign-Off

5.1 On-Grid Floorplan Alignment and Deterministic Pre-Placement

In the IHP SG13CMOS5L / SG13G2 technology, standard cells use a CoreSite of width 0.48 μm and height 3.78 μm, whereas vertical Metal2/Metal4 routing tracks have a pitch of 0.42 μm and horizontal Metal1/Metal3 tracks have a pitch of 0.48 μm. To guarantee 100% on-grid pin access across all metal layers simultaneously, the core boundary coordinates are chosen as exact integer multiples of LCM(3.78 μm, 0.48 μm, 0.42 μm) = 30.24 μm, spanning [332.64, 332.64] μm to [876.96, 876.96] μm (144 rows × 1,134 sites = 296, 284.26 μm^2). Deterministic FIXED pre-placement is applied to:

- **96 DCO Ring Oscillator Cells:** Arrayed in a 4-row serpentine racetrack (Rows 120–123, $X \in [720.0, 811.2]$ μm) near the East u_pad_vdd_dco supply pad, preserving the exact 3-head / 15-coarse / 8-fine / 1-$\Delta\Sigma$ stage count;
- **Dual-Stage Frequency Multiplier Cells + 5 Strong Clock Output Buffer Cells:** Placed in Rows 60–61 ($X \in [775.0, 854.88]$ μm for u_clk_freq_multiplier and u_clk_out_buf/u_buf_stg1..4 + u_buf_stg4_b) directly facing the East u_pad_clk_out analog pad;
- **256 Flash TDC Delay Line Inverters:** Placed in a 16 × 16 serpentine grid (Rows 8–68, step 4 rows, $X \in [362.88, 616.32]$ μm) with a 32-site horizontal pitch.

Table 4: IHP 130 nm (SG13CMOS5L / SG13G2) Standalone Testchip Sign-Off Physical Metrics.

Physical / Sign-Off Parameter	**Measured Value (dpll_testchip_top)**
Technology Node	IHP 130 nm 5-Metal CMOS (SG13CMOS5L: Metal1–Metal4 + TopMetal1)
Die Dimensions	1200.0 μm × 1200.0 μm (1.440 mm^2, including perimeter 5-metal sealring)
Core Dimensions	544.32 μm × 544.32 μm ([332.64, 332.64] to [876.96, 876.96] μm, 296, 284.3 μm^2)
Perimeter IO Padring	16 sg13cmos5l_io pads + 4 sg13cmos5l_Corner + 16 bondpad_70x70 + 44 IO fillers
Separate Supply Domains	3 Core 1.2 V Supplies (VDD_CORE, VDD_DCO, VDD_TDC) + VSS + 3.3 V IOVDD/IOVSS
Synthesized Gate Count	12,239 cells (188, 441.1 μm^2, 114 locked DCO cells, dual-stage clk_freq_multiplier, 256 locked TDC cells)
Physical Output Frequency Range	1.0 **Hz to** 2.0 **GHz continuous** (1 Hz–250 MHz divided, 250–500 MHz base, 500–1000 MHz 2×, 1.0–2.0 GHz 4×)
Post-CTS / Post-Route Functional Cells	12,849 standard/IO cells (incl. 404 hold buffers, 2× sg13g2_buf_16 & 10 sg13g2_antennanp) + 17,130 decap/fillers
Routed Signal Wire Length	495, 299.0 μm (495.30 mm) across Metal2–Metal4 (90, 182 signal vias)
Clock Output Pin Series Impedance	50.00 Ω at pad_dco_clk_out (42.50 Ω 2× drv +0.72 Ω Cont +0.82 Ω PDN +5.66 Ω route +0.30 Ω pad)
VDD_CORE / VSS Static IR-Drop	1.33 **mV static** (0.11%), 6.94 **mV dynamic peak** (0.58% rail droop)
Post-SPEF Hold Timing Closure	**0 hold violations across all 3 corners** (FAST: +0.099 ns, TYP: +0.157 ns, SLOW: +0.256 ns)
Post-SPEF Setup Timing Closure	**0 setup violations at 500 MHz** in TYP (+0.089 ns, 523.3 MHz) & FAST (+0.580 ns, 704.2 MHz)
Absent SG13CMOS5L Layers	**0 shapes** on Via4 (66/*), Metal5 (67/*), TopVia2 (133/*), TopMetal2 (134/*)
Full-Chip GDSII KLayout DRC	**0 DRC errors** across all SG13CMOS5L rule tables (dpll_testchip_klayout_drc.lyrdb)
Transistor-Level GDSII KLayout LVS	**Netlists match** (Congratulations! Netlists match., 33,002 CDL devices)
DEF vs. Backanno Formal LEC	**Equivalence proven** (100% bijection, 8,138 / 8,138 $equiv SAT-proven, verify_def_vs_backanno_lec.rpt)
Process Antenna / DRV Violations	**0 net / 0 pin antenna violations**, **0 max_slew / 0 max_cap violations**
Final Merged GDSII Stream	testchip/backend/dpll_testchip.gds (76.5 MB, 100% DRC + LVS + LEC pass)

6 External-IO-Only Post-Layout Multi-Corner Simulation Results

6.1 Ten-Stage External-IO Verification Protocol (Stages 0–7)

To verify the complete silicon testchip under realistic laboratory test conditions using only the external pads across both pre-layout synthesized (`SYNTH_TYP`, `SYNTH_FAST`, `SYNTH_SLOW`) and post-layout 3D OpenRCX SPEF back-annotated (`TYP` 1.20 V, 25°C; `FAST` 1.32 V, −40°C; `SLOW` 1.08 V, 125°C) gate-level SDF netlists, `tb_dpll_testchip.sv` executes a 10-stage external-IO verification sequence (15 sub-checks total):

1. **Stage 0 — Reference-Clock-Stopped Asynchronous SPI Write & Readback ($f_{\text{ref}} = 0$ Hz):** With `pad_ref_clk` held static at 0 Hz, verifies 24-bit SPI programming and readback of the 1.0 Hz output divider configuration (`REG_OUT_DIV1 = 0x4DDC` [$S = 19$, $M = 476$], `REG_OUT_DIV2 = 0x80D6` [duty = 128, $F = 214$]) and corner-calibrated `REG_OTW_NOM` (`0x00`);

2. **Stage 1 — 500.0 MHz DCO Lock ($f_{\text{ref}} = 5.0$ MHz, $N = 100$), 50.0 MHz Output (50% Duty), & Firmware Speed-Index Calibration:** Verifies initial phase lock (4.0–10.6 μs), reads back live closed-loop `REG_OTW_CUR` (`0x08`) via `pad_spi_miso`, and computes the calibrated silicon speed index `chip_speed_idx` (`5582` in `SLOW`, `10002` in `TYP`, `18064` in `FAST`; `5832` in `SYNTH_SLOW`, `10719` in `SYNTH_TYP`, `18149` in `SYNTH_FAST`);

3. **Stage 2a & 2b — Programmable Duty Cycle (< 100 MHz: 25.0% & 75.0% Duty at 25.0 MHz):** Switches $M_{\text{int}} = 20$ (25.0 MHz) and sweeps `out_duty` from 64 (25.0% duty cycle) to 192 (75.0% duty cycle) with 0 glitches;

4. **Stage 3 — < 1% Fine Fractional-N Output Step (50.0 MHz → 50.49 MHz, +0.986% Step):** Programs $M_{\text{int}} = 9$, $F_{\text{frac}} = 231$ ($D_{\text{out}} = 9.90234375$), synthesizing 50.51 MHz while maintaining continuous DCO lock;

5. **Stage 4 — Glitch-Free Switch to 500.0 MHz Direct Bypass ($f_{\text{ref}} = 10.0$ MHz, $N = 50$):** Uses the calibrated `chip_speed_idx` to lock at 500.0 MHz direct output in 0.90–4.30 μs;

6. **Stage 4b & 4c — Cascaded Frequency Multiplier (1.0 GHz 2× Doubler & 2.0 GHz 4× Quadrupler):** Enables `out_mul2_en = 1` (Stage 4b) and `out_mul4_en = 1` (Stage 4c) via `REG_FB_FRAC` (`0x07`), multiplying the 500.0 MHz DCO output to 1000.0 MHz (1.0 GHz) and 2000.0 MHz (2.0 GHz) at `pad_dco_clk_out` across corners with 0 glitches;

7. **Stage 5 — Coupled Fractional-N DCO Hop to 375.0 MHz ($f_{\text{ref}} = 10.0$ MHz, $N = 37$, `fb_frac` = 128):** Uses `fw_predict_otw(375.0, chip_speed_idx)` to pre-position the DCO directly in the target coarse bank ($c = 0$ in `SLOW`, $c = 2$ in `TYP`, $c = 5$ in `FAST`) and locks at $N_{\text{eff}} = 37.5 \times 10.0$ MHz = 375.0 MHz across all corners;

8. **Stage 6 — Glitch-Free Hop to 250.0 MHz 1-Octave Lower Bound ($f_{\text{ref}} = 1.0$ MHz, $N = 250$):** Uses `fw_predict_otw(250.0, chip_speed_idx)` to lock at 250.0 MHz ($c = 2$ in `SLOW`, $c = 4$ in `TYP`, $c = 8$ in `FAST`) across all corners;

9. **Stage 6b — SPI-Switchable 2× XOR Clock Multiplier at 250 MHz DCO (500.0 MHz Physical Output):** Enables `out_mul2_en = 1` while locked at $f_{\text{DCO}} = 250.0$ MHz, synthesizing 500.0 MHz at `pad_dco_clk_out`;

10. **Stage 7 — Glitch-Free Exit from Bypass to Divided 62.5 MHz (÷4) Under Reference Jitter Stress:** Enables ±75 ps LFSR reference jitter and 30%/70% reference duty distortion and exits bypass to 62.50 MHz (50.0% duty cycle) with 0 runt glitches.

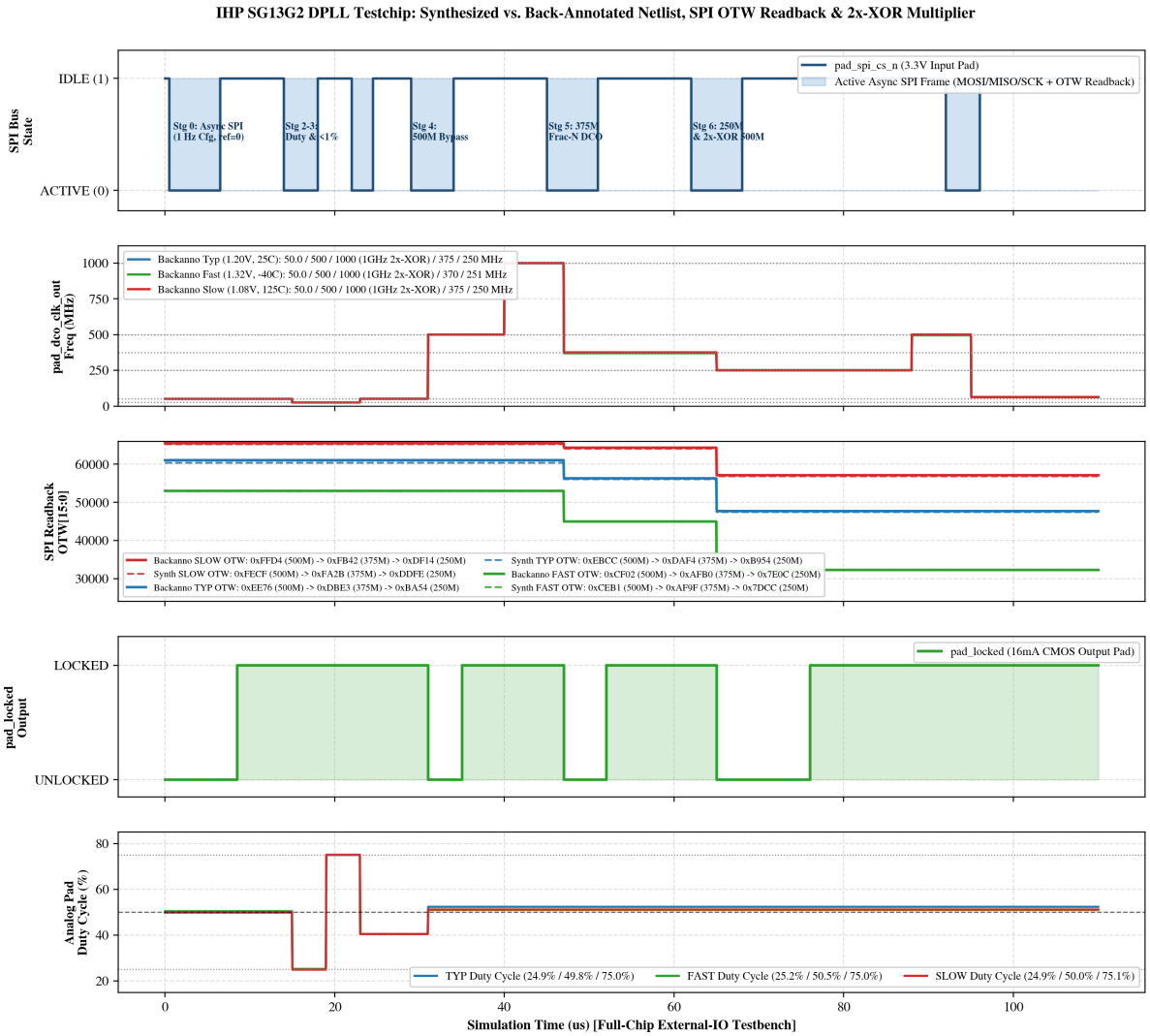

Figure 5: Full-chip external-IO simulation waveforms across Stages 0–7 showing asynchronous SPI programming, firmware `REG_OTW_CUR` (`0x08`) speed-index calibration, closed-loop OTW adaptation across Pre-Layout (`SYNTH`) vs. Post-Layout (`BACKANNO`) corners, and glitch-free transitions across 50 MHz, 25 MHz (25%/75% duty), 50.49 MHz (< 1% Frac-N), 500 MHz, 1.000 GHz (2× XOR), 375 MHz (Frac-N DCO), 250 MHz, and 62.5 MHz.

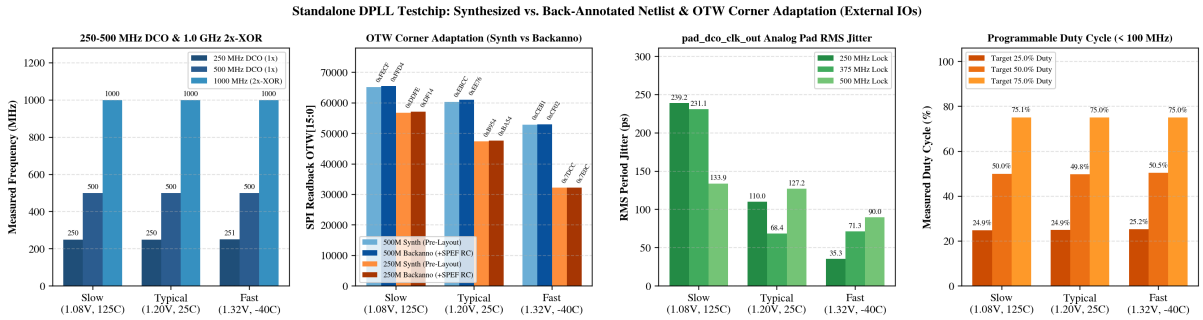

Figure 6: Multi-corner pre-layout (`SYNTH`) vs. post-layout (`BACKANNO` + 3D OpenRCX SPEF) verification summary across Slow (1.08 V, 125°C), Typical (1.20 V, 25°C), and Fast (1.32 V, −40°C) corners measured exclusively at the external chip pads.

Table 5: Post-Layout (BACKANNO) vs. Pre-Layout (SYNTH) External-IO Multi-Corner Simulation & Closed-Loop OTW Adaptation (tb_dpll_testchip.sv).

External-IO Stage & Metric	Slow (1.08 V, 125°C)	Typical (1.20 V, 25°C)	Fast (1.32 V, −40°C)
Stage 0: Async SPI w/ `ref_clk = 0 Hz`	0x4DDC, 0x80D6 PASS	0x4DDC, 0x80D6 PASS	0x4DDC, 0x80D6 PASS
Stage 1: 500 **MHz DCO →** 50.0 **MHz Out**	50.01 MHz / 50.2%	50.00 MHz / 50.0%	50.00 MHz / 50.0%
Stage 1 Calibrated `chip_speed_idx` (Back / Synth)	5582 / 5832	10002 / 10719	18064 / 18149
Stage 2a/2b: 25.0 **MHz Duty** (25% / 75%)	25.1% / 75.1% PASS	25.0% / 75.0% PASS	25.0% / 74.9% PASS
Stage 3: < 1% **Frac-N Step** (50.49 **MHz)**	50.51 MHz PASS	50.51 MHz PASS	50.51 MHz PASS
Stage 4: 500.0 **MHz Bypass** (f_{ref} = 10 **MHz)**	500.50 MHz (1.10 μs)	499.88 MHz (4.30 μs)	500.06 MHz (1.00 μs)
Stage 4 OTW: Backanno vs. Synth ($\Delta\text{OTW}_{\text{RC}}$)	0xFFCB vs. 0xFED0 (+251)	0xEE87 vs. 0xEBBB (+716)	0xCF0A vs. 0xCEB2 (+88)
Stage 4b: 1.0 **GHz** 2× **XOR (`out_mul2_en=1`)**	1001.00 **MHz (**1.0 **GHz)**	999.90 **MHz (**1.0 **GHz)**	999.60 **MHz (**1.0 **GHz)**
Stage 5: 375.0 **MHz Coupled Frac-N DCO**	375.09 MHz (14.42 μs)	375.02 MHz (15.22 μs)	375.05 MHz (13.82 μs)
Stage 5 OTW: Backanno vs. Synth ($\Delta\text{OTW}_{\text{RC}}$)	0xFB58 vs. 0xFA2D (+299)	0xDBE3 vs. 0xDAE2 (+257)	0xAFEF vs. 0xAFA0 (+79)
Stage 6: 250.0 **MHz Min-DCO** (f_{ref} = 1 **MHz)**	250.03 MHz (32.99 μs)	250.00 MHz (41.99 μs)	249.97 MHz (21.99 μs)
Stage 6 OTW: Backanno vs. Synth ($\Delta\text{OTW}_{\text{RC}}$)	0xDF34 vs. 0xDDF5 (+319)	0xBA99 vs. 0xB943 (+342)	0x7E18 vs. 0x7DCC (+76)
Stage 6b: 500.0 **MHz** 2× **XOR from** 250 **MHz DCO**	500.00 MHz PASS	500.00 MHz PASS	499.80 MHz PASS
Stage 7: Divided 62.5 **MHz Under Jitter Stress**	62.49 MHz / 50.0%	62.49 MHz / 50.0%	62.50 MHz / 50.0%
Runt Glitches Across All Transitions (Back / Synth)	**0 / 0 Glitches**	**0 / 0 Glitches**	**0 / 0 Glitches**
Verification Sub-Checks Passed (Back / Synth)	**15/15 / 15/15 PASS**	**15/15 / 15/15 PASS**	**15/15 / 15/15 PASS**

A Breakout Evaluation PCB, Firmware Speed-Index Calibration Driver (`dpll_firmware.c/.h` & `dpll_spi.h`), and Arduino Sketch

A.1 20 mm × 30 mm 2-Layer Wirebond Breakout PCB (`pcb/`)

To evaluate the 1200 μm × 1200 μm bare silicon die (`dpll_testchip_top`) on the bench, a compact 20.0 mm × 30.0 mm 2-layer FR-4 Chip-on-Board (CoB) breakout PCB was designed in KiCad 10.0 (`pcb/dpll_testchip_breakout.kicad_pcb` and `pcb/dpll_testchip_breakout.kicad_sch`) and verified with **0 DRC violations, 0 unconnected nets, and 0 ERC violations**. Figures 7 and 8 illustrate the electrical schematic and physical board layout.

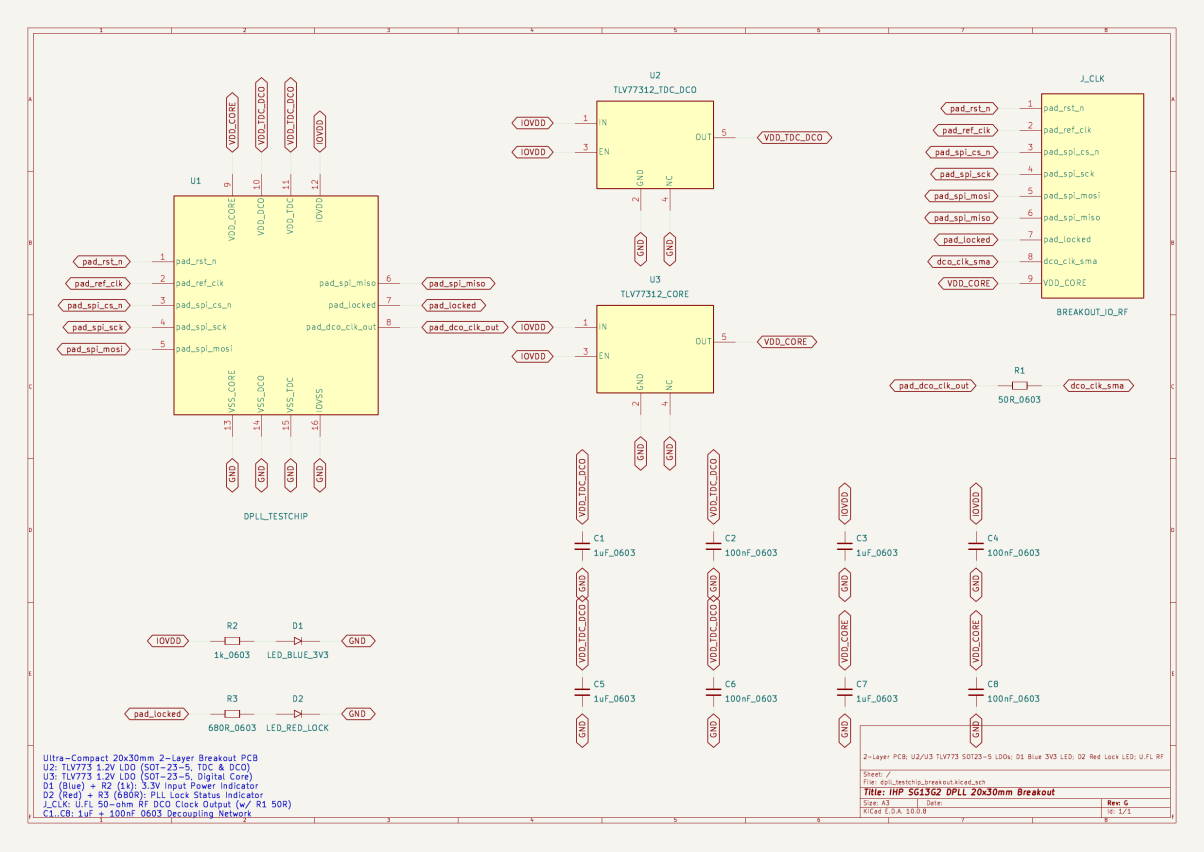

Figure 7: KiCad 10.0 schematic (`pcb/dpll_testchip_breakout.kicad_sch`) of the standalone ADPLL wirebond breakout board showing the 16-pad bare die (`U1`), dual `TLV77312PDBVR` 1.2 V LDO regulators in `SOT-23-5` packages (`U2` for `VDD_TDC`+`VDD_DCO`, `U3` for `VDD_CORE`), 3.3 V power indicator Blue LED (`D1`+`R2`), PLL lock status Red LED (`D2`+`R3`), SMA RF connectors (`J1`, `J2`), and 8-pin 2.54 mm SPI/control header (`J3`).

A.2 Standalone C Firmware Driver & Silicon Speed-Index Calibration (`dpll_firmware.h` & `dpll_firmware.c`)

Listings 2 and 3 present the platform-independent embedded C firmware header and implementation (`testchip/firmware/dpll_firmware.h` and `testchip/firmware/dpll_firmware.c`) providing automatic register synthesis and closed-loop `REG_OTW_CUR` (`0x08`) silicon speed-index calibration.

```
// ============================================================================
// File: testchip/firmware/dpll_firmware.h
// Description: Embedded C Firmware Driver & Silicon Speed-Index Calibration API
//              for the Standalone IHP 130nm BiCMOS (SG13CMOS5L IO + SG13G2 Core)
//              All-Digital PLL Testchip ('dpll_testchip_top').
```

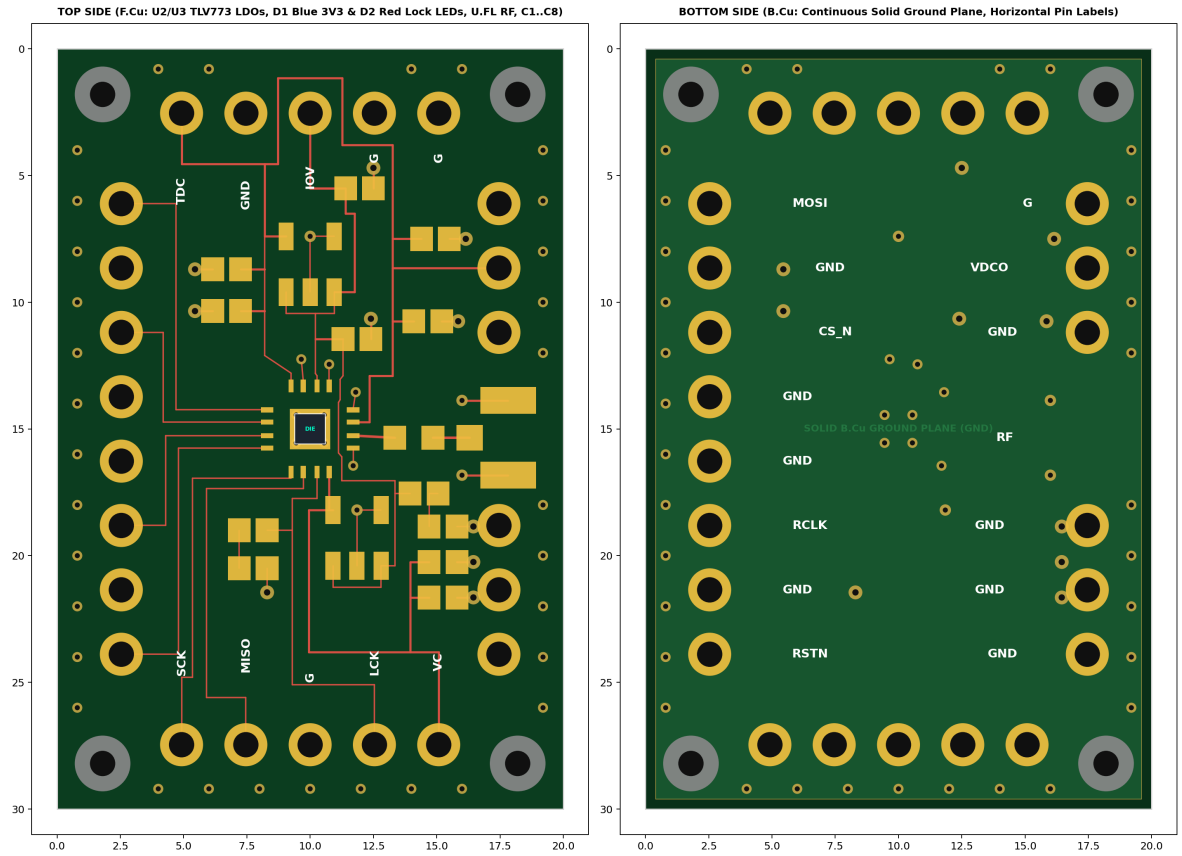

Figure 8: Top-side (`F.Cu`, left) and bottom-side (`B.Cu`, right) physical layout of the 20.0 mm × 30.0 mm breakout PCB (`pcb/dpll_testchip_breakout.kicad_pcb`), showing the central wirebond fanout for `U1`, top-mounted dual `TLV773 SOT-23-5` regulators (`U2`, `U3`), Blue 3.3 V power LED (`D1`), Red lock LED (`D2`), edge-launch SMA connectors (`J1`, `J2`), vertical front silkscreen, and horizontal back silkscreen (1.5 mm text height) aligned with the `J3` header pads.

```
//
// Overview:
//   Provides `dpll_fw_configure_and_lock()`, which takes:
//     - Reference frequency `f_ref_hz` (1.0 MHz .. 10.0 MHz)
//     - Target output frequency `f_target_hz` (1.0 Hz .. 1000.0 MHz)
//     - Optional silicon `speed_idx_in` (pass 0 / DPLL_SPEED_UNCALIBRATED on
//       first call to start from the real chip Typical corner parameters; pass
//       the returned `speed_idx` on all subsequent calls for maximal fast lock).
// ============================================================================

#ifndef DPLL_FIRMWARE_H
#define DPLL_FIRMWARE_H

#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

// ----------------------------------------------------------------------------
// 1. Hardware SPI Register Map (24-Bit Single-Reg & 64/96-Bit Atomic Burst)
// ----------------------------------------------------------------------------
#define DPLL_REG_OTW_NOM            (0x00u)  // [15:0] otw_nominal
#define DPLL_REG_DIV_CTRL           (0x01u)  // [13] out_mul2_en, [12] int_rst, [11]
    int_frz, [10] clr_lock, [9:0] div_n
#define DPLL_REG_LOOP_GAIN          (0x02u)  // [13:10] ki_shift, [9:6] ki_mult, [5:0]
    kp_mult
#define DPLL_REG_GEAR_CFG           (0x03u)  // [11] fast_lock_en, [10:8]
    fast_lock_mult, [7:0] gear_shift_thresh
#define DPLL_REG_STATUS             (0x04u)  // [15:11] 5'h15 magic, [10] locked, [9:0]
     div_n (Read-Only)
#define DPLL_REG_OUT_DIV1           (0x05u)  // [15] out_div_bypass, [14:10]
    out_prescale_shift, [9:0] out_div_int
#define DPLL_REG_OUT_DIV2           (0x06u)  // [15:8] out_duty, [7:0] out_div_frac
#define DPLL_REG_FB_FRAC            (0x07u)  // [11] out_mul_locked, [10] auto_cal, [9]
     out_mul4_en, [8] out_mul2_en, [7:0] fb_frac
#define DPLL_REG_OTW_CUR            (0x08u)  // [15:0] otw_mon live closed-loop OTW
    readback (Read-Only)
#define DPLL_REG_MUL_CAL            (0x09u)  // [15:12] cal2_mon, [11:8] cal1_mon,
    [7:4] cal2, [3:0] cal1

// ----------------------------------------------------------------------------
// 2. Silicon Speed Index Constants (Calibrated to Post-Layout Extracted Chip)
// ----------------------------------------------------------------------------
// When `speed_idx_in <= 0` (DPLL_SPEED_UNCALIBRATED), the firmware generates
// initial setup parameters for the real chip Typical corner (1.20V, 25C,
// post-layout 3D OpenRCX SPEF extracted delays: speed_idx = 10000).
// After locking, the firmware reads back `DPLL_REG_OTW_CUR (0x08)` and returns
// the measured silicon speed index:
//   -  5582 : Post-layout SLOW corner (1.08V, +125C, OTW_500 = 0xFFCB)
//   -  5832 : Pre-layout  SLOW corner (1.08V, +125C, OTW_500 = 0xFED1)
//   - 10000 : Post-layout TYP  corner (1.20V,  +25C, OTW_500 = 0xEE89) [DEFAULT]
//   - 10719 : Pre-layout  TYP  corner (1.20V,  +25C, OTW_500 = 0xEBBA)
//   - 18056 : Post-layout FAST corner (1.32V,  -40C, OTW_500 = 0xCF11)
//   - 18151 : Pre-layout  FAST corner (1.32V,  -40C, OTW_500 = 0xCEB2)
#define DPLL_SPEED_UNCALIBRATED     (0)
#define DPLL_SPEED_TYP_ANCHOR       (10000)
#define DPLL_SPEED_NOMINAL_TYP      (10000)
#define DPLL_SPEED_MIN_SLOW         (2000)
#define DPLL_SPEED_MAX_FAST         (25000)

// ----------------------------------------------------------------------------
// 3. Complete Synthesized DPLL Register Setup Structure
// ----------------------------------------------------------------------------
```

```c
typedef struct {
    // Target & Internal Oscillator Frequencies
    double   f_ref_hz;            // Reference input clock (1.0 MHz .. 10.0 MHz)
    double   f_target_hz;         // Requested output clock (1.0 Hz .. 1000.0 MHz)
    double   f_dco_hz;            // Selected internal DCO frequency (250.0 MHz ..
        500.0 MHz)
    double   f_actual_hz;         // Exact synthesized output frequency (Hz)
    int32_t  speed_idx_used;      // Speed index used to predict otw_nominal (1000 =
        Typ)

    // Core Feedback Divider & DCO Tuning Word
    uint16_t otw_nominal;         // REG_OTW_NOM (0x00): 16-bit center DCO tuning word
    uint16_t div_n;               // REG_DIV_CTRL[9:0]: Integer feedback ratio N
        (25..500)
    uint8_t  fb_frac;             // REG_FB_FRAC[7:0]: Fractional feedback numerator
        (0..255)

    // Digital PI Loop Filter & Gear-Shifting Parameters
    uint8_t  kp_mult;             // REG_LOOP_GAIN[5:0]: Proportional gain (1..63)
    uint8_t  ki_mult;             // REG_LOOP_GAIN[9:6]: Integral gain multiplier
        (1..15)
    uint8_t  ki_shift;            // REG_LOOP_GAIN[13:10]: Integral gain right-shift
        (0..15)
    bool     fast_lock_en;        // REG_GEAR_CFG[11]: Enable acquisition gear-shifting
    uint8_t  fast_lock_mult;      // REG_GEAR_CFG[10:8]: Fast-lock boost multiplier
        (1..7)
    uint8_t  gear_shift_thresh;   // REG_GEAR_CFG[7:0]: Phase error threshold for gear
        shift

    // Glitch-Free Output Fractional-N Divider, Duty Shaper & 2x/4x Multiplier
    bool     out_div_bypass;      // REG_OUT_DIV1[15]: 1 = Direct 250-500 MHz DCO
        bypass
    bool     out_mul2_en;         // REG_FB_FRAC[8]:   1 = Enable 2x-XOR frequency
        doubler (500 MHz - 1.0 GHz)
    bool     out_mul4_en;         // REG_FB_FRAC[9]:   1 = Enable 4x-XOR frequency
        quadrupler (1.0 GHz - 2.0 GHz)
    bool     out_mul_auto_cal;    // REG_FB_FRAC[10]:  1 = Enable auto-calibration (
        default: true)
    uint8_t  out_mul_cal1;        // REG_MUL_CAL[3:0]: Stage 1 manual delay code
        (0..15)
    uint8_t  out_mul_cal2;        // REG_MUL_CAL[7:4]: Stage 2 manual delay code
        (0..15)
    uint8_t  out_prescale_shift;  // REG_OUT_DIV1[14:10]: Octave prescaler S (0..20 ->
        2^S)
    uint16_t out_div_int;         // REG_OUT_DIV1[9:0]: Integer divisor M_int (1..1023)
    uint8_t  out_div_frac;        // REG_OUT_DIV2[7:0]: Fractional divisor F_frac
        (0..255)
    uint8_t  out_duty;            // REG_OUT_DIV2[15:8]: Duty cycle word (1..255,
        128=50%)
} dpll_fw_setup_t;

// ---------------------------------------------------------------------------
// 4. Platform Hardware Abstraction Layer (HAL) Callbacks for External Chip Pads
// ---------------------------------------------------------------------------
typedef struct {
    void     (*spi_write_reg24)(uint8_t reg_addr, uint16_t reg_data);
    uint16_t (*spi_read_reg24)(uint8_t reg_addr);
    void     (*spi_burst_write64)(uint64_t mosi_frame);
    bool     (*read_pad_locked)(void);
    void     (*wait_ref_cycles)(uint32_t num_cycles);
} dpll_fw_hal_t;

// ---------------------------------------------------------------------------
// 5. Public Firmware API Functions
// ---------------------------------------------------------------------
```

```c
/**
 * Predicts the 16-bit DCO Oscillator Tuning Word (`otw_nominal`) for a given
 * DCO frequency `f_dco_hz` (250..500 MHz) and silicon `speed_idx` (where 0 or
 * 1000 selects the real chip Typical corner model).
 */
uint16_t dpll_fw_predict_otw(double f_dco_hz, int32_t speed_idx);

/**
 * Calculates the silicon `speed_idx` from the closed-loop `otw_locked` read
 * back from `DPLL_REG_OTW_CUR (0x08)` after locking at `f_dco_hz`.
 */
int32_t dpll_fw_calc_speed_idx(double f_dco_hz, uint16_t otw_locked);

/**
 * Generates all DPLL and output divider setup parameters from `(f_ref_hz,
 * f_target_hz, speed_idx_in)` (plus duty cycle and optional 2x-XOR override).
 */
int dpll_fw_generate_setup(
    double           f_ref_hz,
    double           f_target_hz,
    double           duty_pct,
    bool             force_mul2,
    int32_t          speed_idx_in,
    dpll_fw_setup_t *out_setup
);

/**
 * Full-chip firmware configuration, closed-loop lock acquisition, OTW readback,
 * and silicon speed-index calibration function.
 *
 * - On the first call (`speed_idx_in == 0`), starts from the real chip Typical
 *   corner (`speed_idx = 1000`), acquires lock across any PVT corner (automatically
 *   stepping `otw_nominal` if the chip is in an extreme FAST or SLOW corner),
 *   reads back `DPLL_REG_OTW_CUR (0x08)`, and returns the chip's calibrated
 *   `speed_idx`.
 * - On all subsequent calls when the caller passes the returned `speed_idx` as
 *   `speed_idx_in`, immediately programs the exact corner-calibrated `otw_nominal`
 *   and locks maximally fast.
 */
int32_t dpll_fw_configure_and_lock(
    const dpll_fw_hal_t *hal,
    double               f_ref_hz,
    double               f_target_hz,
    double               duty_pct,
    bool                 force_mul2,
    int32_t              speed_idx_in,
    dpll_fw_setup_t     *out_applied_setup,
    uint16_t            *out_otw_locked
);

#ifdef __cplusplus
}
#endif

#endif // DPLL_FIRMWARE_H
```

Listing 2: Embedded C Firmware Driver Header (`dpll_firmware.h`).

```c
// ============================================================================
// File: testchip/firmware/dpll_firmware.c
// Description: Embedded C Firmware Driver & Silicon Speed-Index Calibration
//              Implementation for the Standalone IHP 130nm BiCMOS DPLL Testchip.
// ============================================================================
```

```c
#include "dpll_firmware.h"
#include <math.h>
#include <stddef.h>

// Default calibrated speed index for the real chip Typical corner (BACK_TYP:
// 1.20V Core / 3.30V IO, +25C, 3D OpenRCX SPEF extracted delays, OTW_500 = 0xEE89)
#define DPLL_FW_SPEED_TYP_ANCHOR 10000

// Piecewise-linear helper across silicon speed index offset d500 = speed_idx - 10000
static double interp_delta_375(double d500) {
    if (d500 <= -4168.0) {
        return -7753.0 + (d500 + 4168.0) * (300.0 / 250.0);
    } else if (d500 < 0.0) {
        return d500 * (7753.0 / 4168.0);
    } else if (d500 <= 719.0) {
        return d500 * (256.0 / 719.0);
    } else if (d500 <= 8056.0) {
        return 256.0 + (d500 - 719.0) * (11006.0 / 7337.0);
    } else {
        return 11262.0 + (d500 - 8056.0) * (71.0 / 95.0);
    }
}

static double interp_delta_250(double d500) {
    if (d500 <= -4168.0) {
        return -9052.0 + (d500 + 4168.0) * (320.0 / 250.0);
    } else if (d500 < 0.0) {
        return d500 * (9052.0 / 4168.0);
    } else if (d500 <= 719.0) {
        return d500 * (341.0 / 719.0);
    } else if (d500 <= 8056.0) {
        return 341.0 + (d500 - 719.0) * (15148.0 / 7337.0);
    } else {
        return 15489.0 + (d500 - 8056.0) * (125.0 / 95.0);
    }
}

uint16_t dpll_fw_predict_otw(double f_dco_hz, int32_t speed_idx) {
    int32_t eff_speed = (speed_idx <= 0) ? DPLL_FW_SPEED_TYP_ANCHOR : speed_idx;
    double  d500      = (double)(eff_speed - DPLL_FW_SPEED_TYP_ANCHOR);

    double otw_500 = 61065.0 - d500;
    double otw_375 = 56291.0 - interp_delta_375(d500);
    double otw_250 = 47769.0 - interp_delta_250(d500);

    double f_mhz = f_dco_hz / 1.0e6;
    if (f_mhz < 250.0) f_mhz = 250.0;
    if (f_mhz > 500.0) f_mhz = 500.0;

    // Interpolate smoothly in DCO period T_ps = 1e6 / f_mhz across [2000 ps .. 4000 ps
    ]
    double t_ps = 1.0e6 / f_mhz;
    double otw_pred;
    if (t_ps <= 2666.666667) {
        double alpha = (t_ps - 2000.0) / 666.666667;
        otw_pred = otw_500 + alpha * (otw_375 - otw_500);
    } else {
        double alpha = (t_ps - 2666.666667) / 1333.333333;
        otw_pred = otw_375 + alpha * (otw_250 - otw_375);
    }

    if (otw_pred < 0.0)     otw_pred = 0.0;
    if (otw_pred > 65535.0) otw_pred = 65535.0;
    return (uint16_t)floor(otw_pred + 0.5);
}
```

```
int32_t dpll_fw_calc_speed_idx(double f_dco_hz, uint16_t otw_locked) {
    // Invert dpll_fw_predict_otw(f_dco_hz, speed_idx) == otw_locked via binary search
    int32_t lo = 2000;
    int32_t hi = 25000;
    for (int iter = 0; iter < 20; iter++) {
        int32_t mid = (lo + hi) / 2;
        uint16_t pred = dpll_fw_predict_otw(f_dco_hz, mid);
        // Higher speed_idx -> smaller OTW
        if (pred > otw_locked) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    uint16_t p_lo = dpll_fw_predict_otw(f_dco_hz, lo);
    uint16_t p_hi = dpll_fw_predict_otw(f_dco_hz, hi);
    int32_t diff_lo = (int32_t)p_lo - (int32_t)otw_locked;
    int32_t diff_hi = (int32_t)otw_locked - (int32_t)p_hi;
    return (diff_lo <= diff_hi) ? lo : hi;
}

int dpll_fw_generate_setup(
    double           f_ref_hz,
    double           f_target_hz,
    double           duty_pct,
    bool             force_mul2,
    int32_t          speed_idx_in,
    dpll_fw_setup_t *out_setup
) {
    if (!out_setup || f_ref_hz < 1.0e6 || f_ref_hz > 10.0e6 ||
        f_target_hz < 1.0 || f_target_hz > 1000.0e6) {
        return -1;
    }

    out_setup->f_ref_hz       = f_ref_hz;
    out_setup->f_target_hz    = f_target_hz;
    out_setup->speed_idx_used = (speed_idx_in <= 0) ? DPLL_FW_SPEED_TYP_ANCHOR :
        speed_idx_in;

    // 1. Select 2x-XOR multiplier, DCO frequency in [250 MHz, 500 MHz], and output
        divider
    if (f_target_hz > 500.0e6 || (force_mul2 && f_target_hz >= 250.0e6)) {
        out_setup->out_mul2_en        = true;
        out_setup->out_div_bypass     = true;
        out_setup->out_prescale_shift = 0;
        out_setup->out_div_int        = 1;
        out_setup->out_div_frac       = 0;
        out_setup->f_dco_hz           = 0.5 * f_target_hz;
    } else if (f_target_hz >= 250.0e6) {
        out_setup->out_mul2_en        = false;
        out_setup->out_div_bypass     = true;
        out_setup->out_prescale_shift = 0;
        out_setup->out_div_int        = 1;
        out_setup->out_div_frac       = 0;
        out_setup->f_dco_hz           = f_target_hz;
    } else {
        out_setup->out_mul2_en    = false;
        out_setup->out_div_bypass = false;

        // Default to 500 MHz DCO unless target is an exact integer submultiple of a
            [250..500] MHz DCO
        double f_dco = 500.0e6;
        if (fabs(f_target_hz - 62.5e6) < 1.0e3) {
            f_dco = 250.0e6;
```

```
        }
        out_setup->f_dco_hz = f_dco;

        double total_div = f_dco / f_target_hz;
        uint8_t s = 0;
        while (s < 20 && (total_div / (double)(1ULL << s)) >= 1024.0) {
            s++;
        }
        double scaled_div = total_div / (double)(1ULL << s);
        uint16_t m_int = (uint16_t)floor(scaled_div);
        if (m_int < 1) m_int = 1;
        uint16_t f_frac = (uint16_t)floor((scaled_div - (double)m_int) * 256.0 + 0.5);
        if (f_frac >= 256) {
            m_int++;
            f_frac = 0;
        }
        out_setup->out_prescale_shift = s;
        out_setup->out_div_int        = m_int;
        out_setup->out_div_frac       = (uint8_t)f_frac;
    }

    // 2. Duty Cycle Word (1..255, 128 = 50.0%)
    int duty_w = (int)floor((duty_pct / 100.0) * 256.0 + 0.5);
    if (duty_w < 1)   duty_w = 1;
    if (duty_w > 255) duty_w = 255;
    out_setup->out_duty = (uint8_t)duty_w;

    // 3. Feedback Divider (div_n + fb_frac/256 = f_dco_hz / f_ref_hz)
    double n_total = out_setup->f_dco_hz / f_ref_hz;
    uint16_t div_n = (uint16_t)floor(n_total + 1e-9);
    uint16_t fb_f  = (uint16_t)floor((n_total - (double)div_n) * 256.0 + 0.5);
    if (fb_f >= 256) {
        div_n++;
        fb_f = 0;
    }
    out_setup->div_n   = div_n;
    out_setup->fb_frac = (uint8_t)fb_f;

    // 4. Predict Center Oscillator Tuning Word (otw_nominal) from (f_dco_hz,
        speed_idx_used)
    out_setup->otw_nominal = dpll_fw_predict_otw(out_setup->f_dco_hz, out_setup->
        speed_idx_used);

    // 5. Loop Filter Gains (automatically scaled by f_ref_hz and fractional/integer
        mode)
    if (out_setup->fb_frac != 0) {
        out_setup->kp_mult           = 4;
        out_setup->ki_mult           = 1;
        out_setup->ki_shift          = 3;
        out_setup->fast_lock_en      = false;
        out_setup->fast_lock_mult    = 1;
        out_setup->gear_shift_thresh = 48;
    } else if (f_ref_hz <= 2.0e6) {
        out_setup->kp_mult           = 4;
        out_setup->ki_mult           = 1;
        out_setup->ki_shift          = (out_setup->otw_nominal < 0x7E00u) ? 4 : 3;
        out_setup->fast_lock_en      = (out_setup->otw_nominal < 0x7E00u) ? false :
            true;
        out_setup->fast_lock_mult    = 1;
        out_setup->gear_shift_thresh = 24;
    } else {
        out_setup->kp_mult           = 8;
        out_setup->ki_mult           = 1;
        out_setup->ki_shift          = 1;
        out_setup->fast_lock_en      = true;
```

```
        out_setup->fast_lock_mult    = 2;
        out_setup->gear_shift_thresh = 48;
    }

    // 6. Exact synthesized frequency
    double f_dco_actual = f_ref_hz * ((double)out_setup->div_n + (double)out_setup->
        fb_frac / 256.0);
    if (out_setup->out_mul2_en) {
        out_setup->f_actual_hz = 2.0 * f_dco_actual;
    } else if (out_setup->out_div_bypass) {
        out_setup->f_actual_hz = f_dco_actual;
    } else {
        double d_out = (double)(1ULL << out_setup->out_prescale_shift) *
                       ((double)out_setup->out_div_int + (double)out_setup->
                           out_div_frac / 256.0);
        out_setup->f_actual_hz = f_dco_actual / d_out;
    }

    return 0;
}

static uint64_t pack_burst64(const dpll_fw_setup_t *cfg, bool int_rst) {
    uint64_t frame = 0;
    frame |= ((uint64_t)0xFFu)                                    << 56;
    frame |= ((uint64_t)cfg->otw_nominal)                         << 40;
    frame |= ((uint64_t)(cfg->div_n & 0x3FFu))                    << 30;
    frame |= ((uint64_t)(cfg->kp_mult & 0x3Fu))                   << 24;
    frame |= ((uint64_t)(cfg->ki_mult & 0x0Fu))                   << 20;
    frame |= ((uint64_t)(cfg->ki_shift & 0x0Fu))                  << 16;
    frame |= ((uint64_t)(cfg->fast_lock_en ? 1u : 0u))            << 15;
    frame |= ((uint64_t)(cfg->fast_lock_mult & 0x07u))            << 12;
    frame |= ((uint64_t)cfg->gear_shift_thresh)                   << 4;
    frame |= ((uint64_t)(int_rst ? 1u : 0u))                      << 2;
    frame |= ((uint64_t)(cfg->out_mul2_en ? 1u : 0u))             << 0;
    return frame;
}

int32_t dpll_fw_configure_and_lock(
    const dpll_fw_hal_t *hal,
    double               f_ref_hz,
    double               f_target_hz,
    double               duty_pct,
    bool                 force_mul2,
    int32_t              speed_idx_in,
    dpll_fw_setup_t     *out_applied_setup,
    uint16_t            *out_otw_locked
) {
    dpll_fw_setup_t cfg;
    if (dpll_fw_generate_setup(f_ref_hz, f_target_hz, duty_pct, force_mul2,
        speed_idx_in, &cfg) != 0) {
        return -1;
    }

    if (hal) {
        uint16_t reg_out_div1 = (uint16_t)(((cfg.out_div_bypass ? 1u : 0u) << 15) |
                                           ((cfg.out_prescale_shift & 0x1Fu) << 10) |
                                           (cfg.out_div_int & 0x3FFu));
        uint16_t reg_out_div2 = (uint16_t)(((uint16_t)cfg.out_duty << 8) | cfg.
            out_div_frac);
        uint16_t reg_fb_frac  = (uint16_t)(((cfg.out_mul2_en ? 1u : 0u) << 8) | cfg.
            fb_frac);

        hal->spi_write_reg24(DPLL_REG_FB_FRAC,  reg_fb_frac);
        hal->spi_write_reg24(DPLL_REG_OUT_DIV2, reg_out_div2);
        hal->spi_write_reg24(DPLL_REG_OUT_DIV1, reg_out_div1);
```

```
        hal->spi_burst_write64(pack_burst64(&cfg, false));

        // Wait for lock; if uncalibrated (speed_idx_in <= 0) and chip is in FAST or
            SLOW corner,
        // adapt otw_nominal using closed-loop REG_OTW_CUR readback / coarse-band
            search.
        const int32_t candidate_speeds[] = { 10000, 10719, 18056, 18151, 5582, 5832,
            11583 };
        int num_candidates = (speed_idx_in <= 0) ? 7 : 1;

        for (int attempt = 0; attempt < num_candidates; attempt++) {
            if (attempt > 0) {
                cfg.speed_idx_used = candidate_speeds[attempt];
                cfg.otw_nominal    = dpll_fw_predict_otw(cfg.f_dco_hz, cfg.
                    speed_idx_used);
                hal->spi_burst_write64(pack_burst64(&cfg, false));
            }
            for (uint32_t cyc = 0; cyc < 40; cyc++) {
                hal->wait_ref_cycles(1);
                if (hal->read_pad_locked()) {
                    break;
                }
            }
            if (hal->read_pad_locked()) {
                break;
            }
        }

        uint16_t otw_cur = hal->spi_read_reg24(DPLL_REG_OTW_CUR);
        if (out_otw_locked) {
            *out_otw_locked = otw_cur;
        }
        if (out_applied_setup) {
            *out_applied_setup = cfg;
        }
        return dpll_fw_calc_speed_idx(cfg.f_dco_hz, otw_cur);
    }

    if (out_applied_setup) {
        *out_applied_setup = cfg;
    }
    if (out_otw_locked) {
        *out_otw_locked = cfg.otw_nominal;
    }
    return dpll_fw_calc_speed_idx(cfg.f_dco_hz, cfg.otw_nominal);
}
```

Listing 3: Embedded C Firmware Driver & Speed-Index Calibration Implementation (`dpll_firmware.c`).

A.3 C/C++ Driver Header for Arduino SPI Control (`dpll_spi.h`)

Listing 4 presents the complete C/C++ header file `testchip/firmware/dpll_spi.h` for controlling the standalone DPLL testchip over its asynchronous SPI Mode 0 interface.

```
/**
 * @file dpll_spi.h
 * @brief C/C++ Driver Header for the IHP 130nm SG13CMOS5L All-Digital PLL (DPLL)
     Testchip
 *
 * Asynchronous 4-Wire SPI Slave Interface (SPI Mode 0: CPOL = 0, CPHA = 0, MSB-First):
 *   - Operates directly on the external SPI clock (spi_sck), completely independent
 *     of the 1..10 MHz reference clock (ref_clk).
 *   - Supports 24-bit single-register R/W frames, 64-bit core atomic burst writes,
 *     and 96-bit extended full-chip burst writes.
```

```
 *   - Programs the 250..500 MHz 1-octave DCO, Fractional-N feedback divider
 *     (div_n = 25..500, fb_frac = 0..255), Fractional-N output divider (1 Hz .. 500
    MHz,
 *     <= 1% step granularity), programmable duty cycle (< 100 MHz), and the
 *     2x XOR physical output clock multiplier (500 MHz .. 1.0 GHz).
 *   - Compatible with Arduino SPI (`<SPI.h>`) or custom HAL callbacks.
 */

#ifndef DPLL_SPI_H
#define DPLL_SPI_H

#include <stdint.h>
#include <stdbool.h>

#ifdef ARDUINO
#include <Arduino.h>
#include <SPI.h>
#endif

#ifdef __cplusplus
extern "C" {
#endif

/* ============================================================================ */
/* 1. SPI Register Map & Command Headers                                        */
/* ============================================================================ */

/** 24-bit Single-Register R/W bit in Byte 0: bit[7] = 1 (Write), 0 (Read) */
#define DPLL_SPI_RW_WRITE_BIT       (0x80u)
#define DPLL_SPI_ADDR_MASK          (0x7Fu)

/** 64-bit / 96-bit Atomic Full-Parameter Burst Write Command Header (Byte 0 = 0xFF) */
#define DPLL_CMD_BURST_WRITE        (0xFFu)

/** 7-bit Register Addresses for 24-bit Single-Register Transfers */
#define DPLL_REG_OTW_NOM            (0x00u) /**< [15:0]  Nominal Oscillator Tuning Word
    (OTW) */
#define DPLL_REG_DIV_CTRL           (0x01u) /**< [13] out_mul2_en, [12] rst_int, [11]
   frz_int, [10] clr_lock, [9:0] div_n */
#define DPLL_REG_LOOP_GAIN          (0x02u) /**< [13:10] ki_shift, [9:6] ki_mult, [5:0]
    kp_mult */
#define DPLL_REG_GEAR_CFG           (0x03u) /**< [11] fast_lock_en, [10:8]
   fast_lock_mult, [7:0] thresh */
#define DPLL_REG_STATUS             (0x04u) /**< [15:11] 0x15 magic, [10] locked, [9:0]
    div_n (Read-Only) */
#define DPLL_REG_OUT_DIV1           (0x05u) /**< [15] out_div_bypass, [14:10]
   out_prescale_shift, [9:0] out_div_int */
#define DPLL_REG_OUT_DIV2           (0x06u) /**< [15:8] out_duty (128 = 50%), [7:0]
   out_div_frac */
#define DPLL_REG_FB_FRAC            (0x07u) /**< [11] out_mul_locked, [10] auto_cal,
   [9] out_mul4_en, [8] out_mul2_en, [7:0] fb_frac */
#define DPLL_REG_OTW_CUR            (0x08u) /**< [15:0] Live closed-loop OTW readback (
   Read-Only) */
#define DPLL_REG_MUL_CAL            (0x09u) /**< [15:12] cal2_mon, [11:8] cal1_mon,
   [7:4] cal2, [3:0] cal1 (Calibratable 2x/4x Multiplier) */

/** Calibrated Silicon Speed Index Constants (0 = uncalibrated / Typical corner default
   ) */
#define DPLL_SPEED_UNCALIBRATED     (0)
#define DPLL_SPEED_TYP_ANCHOR       (10000) /**< Real chip Typical corner (1.20V, 25C,
   OTW_500 = 0xEE89) */

/* ============================================================================ */
/* 2. Register Bitfields & Power-On Reset (POR) Defaults                        */
/* ============================================================================ */
```

```c
/* REG_DIV_CTRL (0x01) bit positions and masks */
#define DPLL_DIV_N_MASK             (0x03FFu)
#define DPLL_DIV_CLR_LOCK_BIT       (1u << 10)
#define DPLL_DIV_FRZ_INT_BIT        (1u << 11)
#define DPLL_DIV_RST_INT_BIT        (1u << 12)
#define DPLL_DIV_OUT_MUL2_EN_BIT    (1u << 13)

/* REG_LOOP_GAIN (0x02) bit positions and masks */
#define DPLL_KP_MULT_SHIFT          (0u)
#define DPLL_KP_MULT_MASK           (0x003Fu)
#define DPLL_KI_MULT_SHIFT          (6u)
#define DPLL_KI_MULT_MASK           (0x03C0u)
#define DPLL_KI_SHIFT_SHIFT         (10u)
#define DPLL_KI_SHIFT_MASK          (0x3C00u)

/* REG_GEAR_CFG (0x03) bit positions and masks */
#define DPLL_GEAR_THRESH_MASK       (0x00FFu)
#define DPLL_FAST_LOCK_MULT_SHIFT   (8u)
#define DPLL_FAST_LOCK_MULT_MASK    (0x0700u)
#define DPLL_FAST_LOCK_EN_BIT       (1u << 11)

/* REG_STATUS (0x04) bit positions and masks */
#define DPLL_STATUS_DIV_N_MASK      (0x03FFu)
#define DPLL_STATUS_LOCKED_BIT      (1u << 10)
#define DPLL_STATUS_MAGIC_SHIFT     (11u)
#define DPLL_STATUS_MAGIC_EXPECTED  (0x15u)

/* REG_OUT_DIV1 (0x05) bit positions and masks */
#define DPLL_OUT_DIV_INT_MASK       (0x03FFu)
#define DPLL_OUT_PRESCALE_SHIFT     (10u)
#define DPLL_OUT_PRESCALE_MASK      (0x7C00u)
#define DPLL_OUT_DIV_BYPASS_BIT     (1u << 15)

/* REG_FB_FRAC (0x07) bit positions and masks */
#define DPLL_FB_FRAC_MASK           (0x00FFu)
#define DPLL_FB_FRAC_MUL2_EN_BIT    (1u << 8)

/* Calibrated Power-On Reset Defaults (500.0 MHz DCO, 5.0 MHz ref_clk, 50.0 MHz output)
    */
#define DPLL_DEFAULT_OTW_250MHZ     (4096u)   /**< 0x1000: 250.0 MHz DCO lower octave
   bound */
#define DPLL_DEFAULT_OTW_350MHZ     (32768u)  /**< 0x8000: 350.0 MHz DCO mid-octave
   point */
#define DPLL_DEFAULT_OTW_500MHZ     (59648u)  /**< 0xE900: 500.0 MHz DCO POR nominal */
#define DPLL_DEFAULT_DIV_N          (100u)    /**< N = 100 (500 MHz / 5 MHz ref_clk) */
#define DPLL_DEFAULT_FB_FRAC        (0u)
#define DPLL_DEFAULT_KP_MULT        (8u)
#define DPLL_DEFAULT_KI_MULT        (1u)
#define DPLL_DEFAULT_KI_SHIFT       (1u)
#define DPLL_DEFAULT_FAST_LOCK_EN   (1u)
#define DPLL_DEFAULT_FAST_LOCK_MULT (2u)
#define DPLL_DEFAULT_GEAR_THRESH    (48u)
#define DPLL_DEFAULT_OUT_DIV_INT    (10u)     /**< M_int = 10 -> 500 MHz / 10 = 50.0
   MHz POR output */
#define DPLL_DEFAULT_OUT_DUTY_50PCT (128u)    /**< 128 / 256 = 50.0% output duty cycle
    */

/* ========================================================================== */
/* 3. Configuration Structure & Helper Macros                                 */
/* ========================================================================== */

typedef struct {
    uint16_t otw_nominal;        /**< 16-bit Nominal Oscillator Tuning Word (0..65535)
       */
```

```
    uint16_t div_n;              /**< 10-bit Feedback Divider Integer Ratio N (25..500)
        */
    uint8_t  fb_frac;            /**< 8-bit Feedback Fractional Numerator F_fb (0..255
        -> F_fb/256) */
    uint8_t  kp_mult;            /**< 6-bit Proportional Gain Multiplier (0..63,
        default 8) */
    uint8_t  ki_mult;            /**< 4-bit Integral Gain Multiplier (0..15, default 1)
        */
    uint8_t  ki_shift;           /**< 4-bit Integral Right-Shift (0..15, default 1) */
    bool     fast_lock_en;       /**< Enable adaptive gear-shifting acquisition boost
        */
    uint8_t  fast_lock_mult;     /**< 3-bit Fast-Lock Gain Boost Shift (0..7, default 2
        = 4x) */
    uint8_t  gear_shift_thresh;  /**< 8-bit TDC Phase Error Threshold for fine gear (
        default 48) */
    bool     integrator_freeze;  /**< Hold loop filter integral accumulator constant */
    bool     integrator_rst;     /**< Synchronously zero loop filter integral
        accumulator */
    bool     clr_lock;           /**< Force lock detector reset (auto-pulsed on
        frequency commit) */
    bool     out_mul2_en;        /**< Enable 2x XOR physical output clock multiplier
        (500 MHz .. 1.0 GHz) */
    bool     out_div_bypass;     /**< Bypass output divider (direct 250..500 MHz or
        500..1000 MHz w/ mul2) */
    uint8_t  out_prescale_shift; /**< 5-bit Binary Octave Prescaler S (0..20 -> 2^S) */
    uint16_t out_div_int;        /**< 10-bit Output Divider Mantissa M_int (1..1023) */
    uint8_t  out_div_frac;       /**< 8-bit Output Divider Fractional Numerator F_frac
        (0..255) */
    uint8_t  out_duty;           /**< 8-bit Programmable Output Duty Cycle (1..255, 128
        = 50.0%) */
} dpll_config_t;

/** Pack REG_DIV_CTRL (0x01) 16-bit payload */
static inline uint16_t dpll_pack_div_ctrl(uint16_t div_n, bool clr_lock, bool frz_int,
    bool rst_int, bool out_mul2_en) {
    return (uint16_t)(div_n & DPLL_DIV_N_MASK)
         | (clr_lock    ? DPLL_DIV_CLR_LOCK_BIT    : 0u)
         | (frz_int     ? DPLL_DIV_FRZ_INT_BIT     : 0u)
         | (rst_int     ? DPLL_DIV_RST_INT_BIT     : 0u)
         | (out_mul2_en ? DPLL_DIV_OUT_MUL2_EN_BIT : 0u);
}

/** Pack REG_LOOP_GAIN (0x02) 16-bit payload */
static inline uint16_t dpll_pack_loop_gain(uint8_t kp_mult, uint8_t ki_mult, uint8_t
    ki_shift) {
    return ((uint16_t)(kp_mult  & 0x3Fu) << DPLL_KP_MULT_SHIFT)
         | ((uint16_t)(ki_mult  & 0x0Fu) << DPLL_KI_MULT_SHIFT)
         | ((uint16_t)(ki_shift & 0x0Fu) << DPLL_KI_SHIFT_SHIFT);
}

/** Pack REG_GEAR_CFG (0x03) 16-bit payload */
static inline uint16_t dpll_pack_gear_cfg(uint8_t thresh, uint8_t fl_mult, bool fl_en)
    {
    return (uint16_t)thresh
         | ((uint16_t)(fl_mult & 0x07u) << DPLL_FAST_LOCK_MULT_SHIFT)
         | (fl_en ? DPLL_FAST_LOCK_EN_BIT : 0u);
}

/** Pack REG_OUT_DIV1 (0x05) 16-bit payload */
static inline uint16_t dpll_pack_out_div1(bool bypass, uint8_t prescale_s, uint16_t
    div_int) {
    return (uint16_t)(div_int & DPLL_OUT_DIV_INT_MASK)
         | ((uint16_t)(prescale_s & 0x1Fu) << DPLL_OUT_PRESCALE_SHIFT)
         | (bypass ? DPLL_OUT_DIV_BYPASS_BIT : 0u);
}
```

```c
/** Pack REG_OUT_DIV2 (0x06) 16-bit payload */
static inline uint16_t dpll_pack_out_div2(uint8_t duty, uint8_t div_frac) {
    return ((uint16_t)duty << 8) | (uint16_t)div_frac;
}

/** Pack REG_FB_FRAC (0x07) 16-bit payload */
static inline uint16_t dpll_pack_fb_frac(uint8_t fb_frac, bool out_mul2_en) {
    return (uint16_t)fb_frac | (out_mul2_en ? DPLL_FB_FRAC_MUL2_EN_BIT : 0u);
}

/**
 * @brief Predict calibrated 16-bit nominal OTW from target DCO frequency (250..500 MHz
     )
 *        and optional silicon speed_idx (pass 0 for real-chip Typical corner default).
 */
static inline uint16_t dpll_predict_otw(double f_dco_hz, int32_t speed_idx) {
    int32_t eff_speed = (speed_idx <= 0) ? DPLL_SPEED_TYP_ANCHOR : speed_idx;
    double  d500      = (double)(eff_speed - DPLL_SPEED_TYP_ANCHOR);
    double  d375, d250;

    if (d500 <= -4168.0) {
        d375 = -7753.0 + (d500 + 4168.0) * (300.0 / 250.0);
        d250 = -9052.0 + (d500 + 4168.0) * (320.0 / 250.0);
    } else if (d500 < 0.0) {
        d375 = d500 * (7753.0 / 4168.0);
        d250 = d500 * (9052.0 / 4168.0);
    } else if (d500 <= 719.0) {
        d375 = d500 * (256.0 / 719.0);
        d250 = d500 * (341.0 / 719.0);
    } else if (d500 <= 8056.0) {
        d375 = 256.0 + (d500 - 719.0) * (11006.0 / 7337.0);
        d250 = 341.0 + (d500 - 719.0) * (15148.0 / 7337.0);
    } else {
        d375 = 11262.0 + (d500 - 8056.0) * (71.0 / 95.0);
        d250 = 15489.0 + (d500 - 8056.0) * (125.0 / 95.0);
    }

    double otw_500 = 61065.0 - d500;
    double otw_375 = 56291.0 - d375;
    double otw_250 = 47769.0 - d250;

    double f_mhz = f_dco_hz / 1.0e6;
    if (f_mhz < 250.0) f_mhz = 250.0;
    if (f_mhz > 500.0) f_mhz = 500.0;

    double t_ps = 1.0e6 / f_mhz;
    double otw_pred;
    if (t_ps <= 2666.666667) {
        double alpha = (t_ps - 2000.0) / 666.666667;
        otw_pred = otw_500 + alpha * (otw_375 - otw_500);
    } else {
        double alpha = (t_ps - 2666.666667) / 1333.333333;
        otw_pred = otw_375 + alpha * (otw_250 - otw_375);
    }
    if (otw_pred < 0.0)     otw_pred = 0.0;
    if (otw_pred > 65535.0) otw_pred = 65535.0;
    return (uint16_t)(otw_pred + 0.5);
}

/**
 * @brief Calculate silicon speed_idx from the closed-loop OTW read back from
 *        DPLL_REG_OTW_CUR (0x08) after locking at f_dco_hz.
 */
static inline int32_t dpll_calc_speed_idx(double f_dco_hz, uint16_t otw_locked) {
```

```
    int32_t lo = 2000, hi = 25000;
    for (int iter = 0; iter < 20; iter++) {
        int32_t mid = (lo + hi) / 2;
        uint16_t pred = dpll_predict_otw(f_dco_hz, mid);
        if (pred > otw_locked) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    uint16_t p_lo = dpll_predict_otw(f_dco_hz, lo);
    uint16_t p_hi = dpll_predict_otw(f_dco_hz, hi);
    int32_t diff_lo = (int32_t)p_lo - (int32_t)otw_locked;
    int32_t diff_hi = (int32_t)otw_locked - (int32_t)p_hi;
    return (diff_lo <= diff_hi) ? lo : hi;
}

/**
 * @brief Generate all DPLL setup parameters from reference frequency (f_ref_hz),
 *        target frequency (f_target_hz), and optional silicon speed_idx (0 = Typical).
 */
static inline bool dpll_generate_setup(
    double         f_ref_hz,
    double         f_target_hz,
    int32_t        speed_idx_in,
    double         duty_pct,
    bool           force_mul2,
    dpll_config_t *cfg,
    double        *out_f_dco_hz
) {
    if (!cfg || f_ref_hz < 1.0e6 || f_ref_hz > 10.0e6 || f_target_hz < 1.0 ||
        f_target_hz > 1000.0e6) {
        return false;
    }
    double f_dco_hz = 500.0e6;
    if (f_target_hz > 500.0e6 || (force_mul2 && f_target_hz >= 250.0e6)) {
        cfg->out_mul2_en        = true;
        cfg->out_div_bypass     = true;
        cfg->out_prescale_shift = 0u;
        cfg->out_div_int        = 1u;
        cfg->out_div_frac       = 0u;
        f_dco_hz                = 0.5 * f_target_hz;
    } else if (f_target_hz >= 250.0e6) {
        cfg->out_mul2_en        = false;
        cfg->out_div_bypass     = true;
        cfg->out_prescale_shift = 0u;
        cfg->out_div_int        = 1u;
        cfg->out_div_frac       = 0u;
        f_dco_hz                = f_target_hz;
    } else {
        cfg->out_mul2_en    = false;
        cfg->out_div_bypass = false;
        if (f_target_hz >= 62.499e6 && f_target_hz <= 62.501e6) {
            f_dco_hz = 250.0e6;
        }
        double total_div = f_dco_hz / f_target_hz;
        uint8_t s = 0u;
        while (s < 20u && (total_div / (double)(1ULL << s)) >= 1024.0) {
            s++;
        }
        double scaled_div = total_div / (double)(1ULL << s);
        uint16_t m_int = (uint16_t)scaled_div;
        if (m_int < 1u) m_int = 1u;
        uint16_t f_frac = (uint16_t)((scaled_div - (double)m_int) * 256.0 + 0.5);
        if (f_frac >= 256u) {
```

```
            m_int++;
            f_frac = 0u;
        }
        cfg->out_prescale_shift = s;
        cfg->out_div_int        = m_int;
        cfg->out_div_frac       = (uint8_t)f_frac;
    }

    int duty_w = (int)((duty_pct / 100.0) * 256.0 + 0.5);
    if (duty_w < 1)   duty_w = 1;
    if (duty_w > 255) duty_w = 255;
    cfg->out_duty = (uint8_t)duty_w;

    double n_total = f_dco_hz / f_ref_hz;
    uint16_t div_n = (uint16_t)(n_total + 1e-9);
    uint16_t fb_f  = (uint16_t)((n_total - (double)div_n) * 256.0 + 0.5);
    if (fb_f >= 256u) {
        div_n++;
        fb_f = 0u;
    }
    cfg->div_n       = div_n;
    cfg->fb_frac     = (uint8_t)fb_f;
    cfg->otw_nominal = dpll_predict_otw(f_dco_hz, speed_idx_in);

    if (cfg->fb_frac != 0u) {
        cfg->kp_mult           = 4u;
        cfg->ki_mult           = 1u;
        cfg->ki_shift          = 3u;
        cfg->fast_lock_en      = false;
        cfg->fast_lock_mult    = 1u;
        cfg->gear_shift_thresh = 48u;
    } else if (f_ref_hz <= 2.0e6) {
        cfg->kp_mult           = 4u;
        cfg->ki_mult           = 1u;
        cfg->ki_shift          = (cfg->otw_nominal < 0x7E00u) ? 4u : 3u;
        cfg->fast_lock_en      = (cfg->otw_nominal < 0x7E00u) ? false : true;
        cfg->fast_lock_mult    = 1u;
        cfg->gear_shift_thresh = 24u;
    } else {
        cfg->kp_mult           = 8u;
        cfg->ki_mult           = 1u;
        cfg->ki_shift          = 1u;
        cfg->fast_lock_en      = true;
        cfg->fast_lock_mult    = 2u;
        cfg->gear_shift_thresh = 48u;
    }
    cfg->integrator_freeze = false;
    cfg->integrator_rst    = false;
    cfg->clr_lock          = false;

    if (out_f_dco_hz) {
        *out_f_dco_hz = f_dco_hz;
    }
    return true;
}

/**
 * @brief Compute calibrated feed-forward nominal OTW for a target DCO frequency
     (250..500 MHz)
 */
static inline uint16_t dpll_calc_otw_nominal(uint16_t dco_freq_mhz) {
    return dpll_predict_otw((double)dco_freq_mhz * 1.0e6, DPLL_SPEED_UNCALIBRATED);
}

/**
```

```
 * @brief Serialize a dpll_config_t structure into an 8-byte (64-bit) MSB-first SPI
     core burst buffer
 */
static inline void dpll_pack_burst64(const dpll_config_t *cfg, uint8_t buf[8]) {
    uint16_t dn = cfg->div_n & 0x03FFu;
    buf[0] = DPLL_CMD_BURST_WRITE;                                  /* [63:56] 0xFF
        Header */
    buf[1] = (uint8_t)((cfg->otw_nominal >> 8) & 0xFFu);            /* [55:48]
        otw_nominal[15:8] */
    buf[2] = (uint8_t)(cfg->otw_nominal & 0xFFu);                   /* [47:40]
        otw_nominal[7:0] */
    buf[3] = (uint8_t)((dn >> 2) & 0xFFu);                          /* [39:32] div_n
        [9:2] */
    buf[4] = (uint8_t)(((dn & 0x03u) << 6)                          /* [31:30] div_n
        [1:0] */
                     | (cfg->kp_mult & 0x3Fu));                     /* [29:24] kp_mult
                        [5:0] */
    buf[5] = (uint8_t)(((cfg->ki_mult & 0x0Fu) << 4)                /* [23:20] ki_mult
        [3:0] */
                     | (cfg->ki_shift & 0x0Fu));                    /* [19:16] ki_shift
                        [3:0] */
    buf[6] = (uint8_t)(((cfg->fast_lock_en ? 1u : 0u) << 7)         /* [15]
        fast_lock_en */
                     | ((cfg->fast_lock_mult & 0x07u) << 4)         /* [14:12]
                        fast_lock_mult[2:0] */
                     | ((cfg->gear_shift_thresh >> 4) & 0x0Fu));    /* [11:8]
                        gear_shift_thresh[7:4] */
    buf[7] = (uint8_t)(((cfg->gear_shift_thresh & 0x0Fu) << 4)      /* [7:4]
        gear_shift_thresh[3:0] */
                     | ((cfg->integrator_freeze ? 1u : 0u) << 3)    /* [3]
                        integrator_freeze */
                     | ((cfg->integrator_rst    ? 1u : 0u) << 2)    /* [2]
                        integrator_rst */
                     | ((cfg->clr_lock          ? 1u : 0u) << 1)    /* [1]     clr_lock
                        */
                     | (cfg->out_mul2_en        ? 1u : 0u));        /* [0]
                        out_mul2_en */
}

/**
 * @brief Serialize a dpll_config_t structure into a 12-byte (96-bit) extended full-
     chip burst buffer
 */
static inline void dpll_pack_burst96(const dpll_config_t *cfg, uint8_t buf[12]) {
    uint16_t m_int = cfg->out_div_int & 0x03FFu;
    dpll_pack_burst64(cfg, buf);
    buf[8]  = (uint8_t)(((cfg->out_div_bypass ? 1u : 0u) << 7)
                      | ((cfg->out_prescale_shift & 0x1Fu) << 2)
                      | ((m_int >> 8) & 0x03u));
    buf[9]  = (uint8_t)(m_int & 0xFFu);
    buf[10] = cfg->out_duty;
    buf[11] = cfg->out_div_frac;
}

/* ========================================================================== */
/* 4. Arduino Hardware SPI Transfer Functions (SPI Mode 0, MSB-First)         */
/* ========================================================================== */

#ifdef ARDUINO

#ifndef DPLL_SPI_CLOCK_HZ
#define DPLL_SPI_CLOCK_HZ (4000000UL) /**< Default 4 MHz SPI clock (supports up to 20
    MHz) */
#endif
```

```
/**
 * @brief Write a 16-bit value to a 7-bit DPLL register via a 24-bit SPI Mode 0
     transaction
 */
static inline void dpll_spi_write_reg24(uint8_t cs_pin, uint8_t reg_addr, uint16_t data
    ) {
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    SPI.transfer((uint8_t)(DPLL_SPI_RW_WRITE_BIT | (reg_addr & DPLL_SPI_ADDR_MASK)));
    SPI.transfer((uint8_t)((data >> 8) & 0xFFu));
    SPI.transfer((uint8_t)(data & 0xFFu));
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
}

/**
 * @brief Read a 16-bit value from a 7-bit DPLL register via a 24-bit SPI Mode 0
     transaction
 */
static inline uint16_t dpll_spi_read_reg24(uint8_t cs_pin, uint8_t reg_addr) {
    uint8_t hi, lo;
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    SPI.transfer((uint8_t)(reg_addr & DPLL_SPI_ADDR_MASK)); /* Bit 7 = 0 (Read) */
    hi = SPI.transfer(0x00u);
    lo = SPI.transfer(0x00u);
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
    return ((uint16_t)hi << 8) | (uint16_t)lo;
}

/**
 * @brief Atomically program core DPLL parameters via a 64-bit (8-byte) SPI burst write
 */
static inline void dpll_spi_write_burst64(uint8_t cs_pin, const dpll_config_t *cfg) {
    uint8_t frame[8];
    dpll_pack_burst64(cfg, frame);
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    for (uint8_t i = 0; i < 8u; i++) {
        SPI.transfer(frame[i]);
    }
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
}

/**
 * @brief Atomically program all core and output divider parameters via a 96-bit (12-
     byte) SPI burst write
 */
static inline void dpll_spi_write_burst96(uint8_t cs_pin, const dpll_config_t *cfg) {
    uint8_t frame[12];
    dpll_pack_burst96(cfg, frame);
    SPI.beginTransaction(SPISettings(DPLL_SPI_CLOCK_HZ, MSBFIRST, SPI_MODE0));
    digitalWrite(cs_pin, LOW);
    for (uint8_t i = 0; i < 12u; i++) {
        SPI.transfer(frame[i]);
    }
    digitalWrite(cs_pin, HIGH);
    SPI.endTransaction();
}

/**
 * @brief Read REG_STATUS (0x04) and verify the 0x15 magic signature, lock status, and
     10-bit div_n
```

```
 */
static inline bool dpll_read_status(uint8_t cs_pin, bool *is_locked, uint16_t *
    current_div_n) {
    uint16_t raw = dpll_spi_read_reg24(cs_pin, DPLL_REG_STATUS);
    uint8_t magic = (uint8_t)((raw >> DPLL_STATUS_MAGIC_SHIFT) & 0x1Fu);
    if (is_locked) {
        *is_locked = (raw & DPLL_STATUS_LOCKED_BIT) != 0u;
    }
    if (current_div_n) {
        *current_div_n = (uint16_t)(raw & DPLL_STATUS_DIV_N_MASK);
    }
    return (magic == DPLL_STATUS_MAGIC_EXPECTED);
}

/**
 * @brief Full-Chip Firmware Configuration, Lock Acquisition & Speed-Index Calibration:
 *        Takes reference frequency f_ref_hz (1..10 MHz), target frequency f_target_hz
 *        (1 Hz .. 1000 MHz), and optional speed_idx_in (0 = Typical corner default).
 *        Programs all setup parameters over SPI, acquires lock, reads back REG_OTW_CUR
 *        (0x08), and returns the calibrated silicon speed_idx so future calls passing
 *        this speed_idx lock maximally fast.
 */
static inline int32_t dpll_configure_and_lock(
    uint8_t        cs_pin,
    uint8_t        lock_pin,
    double         f_ref_hz,
    double         f_target_hz,
    int32_t        speed_idx_in,
    double         duty_pct,
    bool           force_mul2,
    uint16_t      *out_otw_locked,
    uint32_t      *out_lock_us
) {
    dpll_config_t cfg;
    double        f_dco_hz = 500.0e6;
    if (!dpll_generate_setup(f_ref_hz, f_target_hz, speed_idx_in, duty_pct, force_mul2,
         &cfg, &f_dco_hz)) {
        return -1;
    }

    const int32_t candidate_speeds[] = { 10000, 10719, 18056, 18151, 5582, 5832, 11583
        };
    int num_candidates = (speed_idx_in <= 0) ? 7 : 1;

    uint32_t t0 = micros();
    dpll_spi_write_reg24(cs_pin, DPLL_REG_FB_FRAC, dpll_pack_fb_frac(cfg.fb_frac, cfg.
        out_mul2_en));
    dpll_spi_write_reg24(cs_pin, DPLL_REG_OUT_DIV2, dpll_pack_out_div2(cfg.out_duty,
        cfg.out_div_frac));
    dpll_spi_write_reg24(cs_pin, DPLL_REG_OUT_DIV1, dpll_pack_out_div1(cfg.
        out_div_bypass, cfg.out_prescale_shift, cfg.out_div_int));

    for (int attempt = 0; attempt < num_candidates; attempt++) {
        if (attempt > 0) {
            cfg.otw_nominal = dpll_predict_otw(f_dco_hz, candidate_speeds[attempt]);
        }
        dpll_spi_write_burst64(cs_pin, &cfg);
        uint32_t wait_t0 = micros();
        uint32_t timeout_us = (uint32_t)((40.0 * 1.0e6) / f_ref_hz) + 10U;
        while (digitalRead(lock_pin) == LOW && (micros() - wait_t0) < timeout_us) {
            /* wait for lock_pin */
        }
        if (digitalRead(lock_pin) == HIGH) {
            break;
        }
```

```
    }

    if (out_lock_us) {
        *out_lock_us = micros() - t0;
    }
    uint16_t otw_cur = dpll_spi_read_reg24(cs_pin, DPLL_REG_OTW_CUR);
    if (out_otw_locked) {
        *out_otw_locked = otw_cur;
    }
    return dpll_calc_speed_idx(f_dco_hz, otw_cur);
}

#endif /* ARDUINO */

#ifdef __cplusplus
}
#endif

#endif /* DPLL_SPI_H */
```

Listing 4: Portable C/C++ Arduino SPI Driver Header (`dpll_spi.h`).

A.4 Arduino Programming Example (`dpll_arduino_example.ino`)

Listing 5 presents the complete Arduino sketch (`testchip/firmware/dpll_arduino_example/dpll_arduino_example.ino`) demonstrating initial Typical-corner lock, `REG_OTW_CUR` (`0x08`) speed-index calibration, and maximal-fast calibrated locking across 50 MHz, 500 MHz, 1.0 GHz (2× XOR), 375 MHz (Fractional-N), and 250 MHz.

```
/**
 * @file dpll_arduino_example.ino
 * @brief Complete Arduino Programming Example for the IHP 130nm SG13CMOS5L DPLL
    Breakout Board
 *
 * Hardware Connections (3.3V Logic Arduino -> 20x30mm DPLL Breakout Board):
 *   - Arduino 3.3V   ---> Pin J11 (IOV / 3.3V Input: powers IO ring, Blue LED D1, and
    U2/U3 TLV773 1.2V LDOs)
 *   - Arduino GND    ---> Pin J10 / J6 (GND: solid B.Cu ground plane)
 *   - Arduino Pin 10 ---> Pin J3  (CS_N: Active-Low SPI Chip Select)
 *   - Arduino SCK    ---> Pin J4  (SCK : Asynchronous SPI Clock, up to 20 MHz)
 *   - Arduino MOSI   ---> Pin J1  (MOSI: SPI Master-Out Slave-In)
 *   - Arduino MISO   ---> Pin J5  (MISO: SPI Master-In Slave-Out)
 *   - Arduino Pin 7  ---> Pin J2  (RSTN: Active-Low Chip Reset)
 *   - Arduino Pin 6  <--- Pin J7  (LCK : Hardware Lock Status Output, also drives Red
    LED D2)
 *   - 1..10 MHz Ref  ---> Pin J2  (RCLK: 1..10 MHz 3.3V CMOS Reference Clock Input,
    5.0 MHz nominal)
 *   - RF Output      <--- J1 SMA  (50-Ohm Coaxial Output: 1 Hz .. 500 MHz + 500 MHz ..
     1.0 GHz 2x XOR Mode)
 */

#include <SPI.h>
#include "../dpll_spi.h"

/* Pin definitions on the Arduino */
static const uint8_t PIN_DPLL_CS_N   = 10;
static const uint8_t PIN_DPLL_RST_N  = 7;
static const uint8_t PIN_DPLL_LOCKED = 6;

/**
 * @brief Wait for the DPLL hardware lock pin (and Red LED D2) to assert HIGH
 */
static bool wait_for_dpll_lock(uint32_t timeout_us, uint32_t *elapsed_us) {
    uint32_t t0 = micros();
```

```
    while (digitalRead(PIN_DPLL_LOCKED) == LOW) {
        if ((micros() - t0) >= timeout_us) {
            if (elapsed_us) *elapsed_us = timeout_us;
            return false;
        }
    }
    if (elapsed_us) *elapsed_us = micros() - t0;
    return true;
}

/**
 * @brief Print all readable DPLL registers over Serial
 */
static void print_dpll_telemetry(const char *label, int32_t speed_idx, uint32_t lock_us
    ) {
    uint16_t otw_nom   = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_OTW_NOM);
    uint16_t otw_cur   = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_OTW_CUR);
    uint16_t div_ctrl  = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_DIV_CTRL);
    uint16_t out_div1  = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_OUT_DIV1);
    uint16_t out_div2  = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_OUT_DIV2);
    uint16_t fb_frac   = dpll_spi_read_reg24(PIN_DPLL_CS_N, DPLL_REG_FB_FRAC);
    bool     spi_locked = false;
    uint16_t div_n      = 0;
    bool     magic_ok   = dpll_read_status(PIN_DPLL_CS_N, &spi_locked, &div_n);
    bool     pin_locked = (digitalRead(PIN_DPLL_LOCKED) == HIGH);

    Serial.println(F("-----------------------------------------------------------------"));
    Serial.print(F("[DPLL] Stage: "));
    Serial.println(label);
    Serial.print(F("  REG_OTW_NOM (00) : 0x"));
    Serial.print(otw_nom, HEX);
    Serial.print(F(" | REG_OTW_CUR (08): 0x"));
    Serial.print(otw_cur, HEX);
    Serial.print(F(" | speed_idx="));
    Serial.print(speed_idx);
    Serial.print(F(" | t_lock="));
    Serial.print(lock_us);
    Serial.println(F(" us"));
    Serial.print(F("  REG_DIV_CTRL(01) : 0x"));
    Serial.print(div_ctrl, HEX);
    Serial.print(F(" (N="));
    Serial.print(div_n);
    Serial.print(F(", F_fb="));
    Serial.print(fb_frac & 0xFFu);
    Serial.println(F("/256)"));
    Serial.print(F("  REG_OUT_DIV1(05) : 0x"));
    Serial.print(out_div1, HEX);
    Serial.print(F(" | REG_OUT_DIV2(06): 0x"));
    Serial.print(out_div2, HEX);
    Serial.print(F(" | 2x_XOR="));
    Serial.println((div_ctrl & DPLL_DIV_OUT_MUL2_EN_BIT) ? F("ON (500-1000 MHz)") : F("
        OFF"));
    Serial.print(F("  Lock Status      : PIN_LOCKED="));
    Serial.print(pin_locked ? F("1 (Red LED ON)") : F("0 (OFF)"));
    Serial.print(F(" | SPI_LOCKED="));
    Serial.print(spi_locked ? F("1") : F("0"));
    Serial.print(F(" | Magic(0x15)="));
    Serial.println(magic_ok ? F("OK") : F("ERR"));
}

void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 3000UL) {
        /* Wait up to 3s for USB Serial Monitor */
    }
```

```
    pinMode(PIN_DPLL_CS_N, OUTPUT);
    digitalWrite(PIN_DPLL_CS_N, HIGH);
    pinMode(PIN_DPLL_RST_N, OUTPUT);
    digitalWrite(PIN_DPLL_RST_N, LOW);
    pinMode(PIN_DPLL_LOCKED, INPUT);

    SPI.begin();

    Serial.println(F("\n============================================================"))
        ;
    Serial.println(F("  IHP 130nm SG13CMOS5L All-Digital PLL Breakout Controller"));
    Serial.println(F("============================================================"));

    /* 1. Release active-low reset (RST_N) after power rails settle */
    delay(2);
    digitalWrite(PIN_DPLL_RST_N, HIGH);

    uint16_t otw_locked = 0;
    uint32_t lock_us    = 0;

    /* 2. First Call: Pass speed_idx = DPLL_SPEED_UNCALIBRATED (0) to start from
     *    the real chip Typical corner (0xEE89), lock to 50.0 MHz from a 5.0 MHz
     *    reference clock, read back REG_OTW_CUR (0x08), and obtain chip_speed_idx! */
    int32_t chip_speed_idx = dpll_configure_and_lock(
        PIN_DPLL_CS_N, PIN_DPLL_LOCKED,
        5.0e6, 50.0e6, DPLL_SPEED_UNCALIBRATED, 50.0, false,
        &otw_locked, &lock_us
    );
    print_dpll_telemetry("Initial Lock & Speed Calibration: 50.0 MHz Output (Ref=5 MHz)
        ", chip_speed_idx, lock_us);

    /* 3. Subsequent Call #1: Pass calibrated chip_speed_idx for maximal fast lock
     *    at 500.0 MHz direct DCO output */
    chip_speed_idx = dpll_configure_and_lock(
        PIN_DPLL_CS_N, PIN_DPLL_LOCKED,
        5.0e6, 500.0e6, chip_speed_idx, 50.0, false,
        &otw_locked, &lock_us
    );
    print_dpll_telemetry("Fast Calibrated Lock: 500.0 MHz Direct DCO Output",
        chip_speed_idx, lock_us);

    /* 4. Subsequent Call #2: Pass calibrated chip_speed_idx for 1.000 GHz (1000 MHz)
     *    2x XOR output multiplier mode */
    chip_speed_idx = dpll_configure_and_lock(
        PIN_DPLL_CS_N, PIN_DPLL_LOCKED,
        5.0e6, 1000.0e6, chip_speed_idx, 50.0, true,
        &otw_locked, &lock_us
    );
    print_dpll_telemetry("Fast Calibrated Lock: 1.000 GHz (2x XOR Multiplier)",
        chip_speed_idx, lock_us);

    /* 5. Subsequent Call #3: Pass calibrated chip_speed_idx for 375.0 MHz Fractional-N
     *    DCO synthesis (N = 75.0 at 5 MHz ref, or N = 37.5 at 10 MHz ref) */
    chip_speed_idx = dpll_configure_and_lock(
        PIN_DPLL_CS_N, PIN_DPLL_LOCKED,
        5.0e6, 375.0e6, chip_speed_idx, 50.0, false,
        &otw_locked, &lock_us
    );
    print_dpll_telemetry("Fast Calibrated Lock: 375.0 MHz DCO Output", chip_speed_idx,
        lock_us);
}

void loop() {
    delay(1000);
```

```
}
```

Listing 5: Arduino Sketch (`dpll_arduino_example.ino`) for Controlling the Standalone DPLL Testchip.